Redes Neuronales II: perceptrón, funciones de activación, backpropagation y clasificación binaria y multiclase con TensorFlow, Keras y PyTorch

Yohann Javier Nikolay Salazar Cardenas

Docente

Nidia Stella Garcia Roa

Universidad de Cundinamarca

Especialización en Inteligencia Artificial

604013/DEEP LEARNING -CONCEPTOS/FU/CAD2202023205/EIAIIPA2026

Repositorio: https://github.com/Yohann0011/Redes-Neuronales-II

2026

Tabla de Contenido

Introducción 4
Fundamentos Conceptuales del Aprendizaje Profundo 5
El Perceptrón y la Neurona Artificial 5
Funciones de Activación 5
Funciones de Pérdida y Entrenamiento Supervisado 6
El Algoritmo de Backpropagation 7
Clasificación Binaria y Clasificación Multiclase 7
TensorFlow, Keras y PyTorch 8
Metodología 8
Entorno de Trabajo 8
Datos y Preprocesamiento 9
Procedimiento por Práctica 10
Resultados 11
Práctica 1: Perceptrón Implementado con NumPy 11
Práctica 2: Red de una Capa y Red Multicapa con Backpropagation 11
Práctica 3: Clasificación Binaria con TensorFlow y Keras 13
Práctica 4: Clasificación Multiclase con TensorFlow y Keras 14
Práctica 5: MNIST con TensorFlow y Keras 15
Práctica 6: MNIST con PyTorch 17
Comparación de Enfoques y Análisis General 19
Conclusiones 21
Referencias Bibliográficas 23
Apéndice A: Código Fuente del Notebook 01 (Perceptrón con NumPy) 25
Apéndice B: Código Fuente del Notebook 02 (Una Capa y Multicapa) 30
Apéndice C: Código Fuente del Notebook 03 (Binaria con Keras) 38

Apéndice D: Código Fuente del Notebook 04 (Multiclase con Keras) 42
Apéndice E: Código Fuente del Notebook 05 (MNIST con Keras) 46
Apéndice F: Código Fuente del Notebook 06 (MNIST con PyTorch) 53

Introducción

En la actividad previa de este curso se construyeron redes neuronales cuyos pesos eran definidos manualmente, es decir, los modelos calculaban salidas pero no las mejoraban con la experiencia, de modo que el propósito de la presente actividad es incorporar el mecanismo que distingue a una red que computa de una red que aprende, a saber, el uso del error de las predicciones para modificar sus propios parámetros, y para ello se integran seis componentes del aprendizaje profundo: las funciones de activación, la función de pérdida, el algoritmo de backpropagation, el descenso de gradiente, el entrenamiento supervisado y los frameworks que automatizan dichos procesos a escala (Trask, 2019).

El trabajo sigue una progresión deliberada, pues primero se resuelve un problema intuitivo y de baja dimensionalidad —clasificar manzanas y bananos a partir de peso, longitud y nivel de amarillo— mediante implementaciones manuales en NumPy, de modo que cada gradiente sea visible y verificable, sobre esa base se automatiza el mismo modelo con TensorFlow y Keras (Gulli et al., 2023) y, finalmente, se replica con PyTorch (Stevens et al., 2020) un problema real de visión por computador, el reconocimiento de dígitos manuscritos del dataset MNIST (LeCun et al., 2010), con diez clases y sin emplear redes convolucionales, tal como lo exige la consigna de la actividad.

La idea central que organiza el documento puede enunciarse así: una red neuronal no aprende por el solo hecho de producir salidas, sino cuando el error de esas salidas se utiliza para corregir sus parámetros, y todos los desarrollos posteriores —optimizadores, arquitecturas, frameworks— son maneras cada vez más eficientes de ejecutar ese mismo ciclo de forward propagation, cálculo de pérdida, backpropagation y actualización; en

coherencia con esa idea, el documento presenta primero el fundamento matemático, después la metodología, luego los resultados obtenidos con sus evidencias y, por último, las conclusiones y las referencias, mientras el código fuente completo de las seis prácticas se incluye en los apéndices A a F y se publica en el repositorio correspondiente (Salazar Cardenas, 2026).

Fundamentos Conceptuales del Aprendizaje Profundo

El Perceptrón y la Neurona Artificial

La unidad básica de los modelos implementados es la neurona artificial, que calcula una combinación lineal ponderada de sus entradas, añade un término de sesgo y aplica una función de activación, y el perceptrón de Rosenblatt (1958) corresponde al caso en que dicha función es un escalón, vale decir, la salida vale uno si la combinación lineal supera el umbral y cero en caso contrario; su regla de aprendizaje modifica los pesos únicamente cuando la predicción es incorrecta, desplazando el hiperplano de decisión en la dirección que corrige el error, y el teorema de convergencia asociado garantiza que, si las clases son linealmente separables, el procedimiento encuentra un hiperplano separador en un número finito de pasos, propiedad que se verifica experimentalmente en la Práctica 1.

En una red multicapa, la salida de cada capa constituye la entrada de la siguiente, lo que permite componer transformaciones no lineales, y en ese esquema los pesos representan la influencia de cada conexión mientras el sesgo desplaza el umbral de activación, siendo ambos los parámetros que el entrenamiento ajusta (Trask, 2019).

Funciones de Activación

Sin una no linealidad, la composición de capas lineales seguiría siendo lineal y la profundidad no añadiría capacidad de representación, por lo que

en esta actividad se emplean cuatro funciones: la función escalón, propia del perceptrón clásico, no es derivable en el umbral y por ello resulta incompatible con el descenso de gradiente; la función sigmoide comprime su entrada al intervalo abierto entre cero y uno, lo que permite interpretar su salida como probabilidad en problemas binarios, y su derivada puede expresarse a partir de su propia salida, aunque su valor máximo de 0.25 provoca desvanecimiento del gradiente en capas profundas; la función ReLU anula las entradas negativas y deja pasar las positivas, con derivada unitaria en la región activa, razón por la cual es la activación estándar de las capas ocultas (He et al., 2015); y finalmente, la función softmax convierte un vector de puntuaciones en una distribución de probabilidad que suma uno, siendo la activación natural de la capa de salida en problemas multiclase (Gulli et al., 2023).

Funciones de Pérdida y Entrenamiento Supervisado

El aprendizaje supervisado dispone de ejemplos etiquetados, pues cada vector de características se acompaña de su clase verdadera y esa etiqueta es la fuente de la señal de error, mientras la función de pérdida resume en un único número qué tan equivocada está la red; para clasificación binaria con salida sigmoide se emplea la entropía cruzada binaria, que penaliza severamente las predicciones seguras y erróneas y cuyo valor de referencia para un modelo que predice 0.5 es 0.6931, y para clasificación multiclase con salida softmax y etiquetas enteras se emplea la entropía cruzada categórica dispersa, cuyo valor inicial esperado con diez clases equiprobables es 2.3026, de modo que observar esos valores en la época cero constituye una verificación de que la implementación es correcta.

El Algoritmo de Backpropagation

Backpropagation es la aplicación sistemática de la regla de la cadena para obtener el gradiente de la pérdida respecto a cada parámetro, propagando la señal de error desde la capa de salida hacia las capas anteriores (Rumelhart et al., 1986), y conviene precisar su naturaleza, ya que no es un algoritmo de aprendizaje en sí mismo sino el procedimiento eficiente que calcula los gradientes que luego consume el optimizador; en una red de una capa con sigmoide y entropía cruzada binaria, el gradiente respecto a la combinación lineal equivale a la diferencia entre la predicción y la etiqueta, y los gradientes de pesos y sesgos se obtienen promediando esa señal ponderada por las entradas, en tanto que en una red multicapa la señal se propaga atravesando cada activación mediante su derivada, de modo que las neuronas inactivas de una capa ReLU no transmiten corrección alguna.

El descenso de gradiente aplica entonces la actualización de cada parámetro restando el producto entre la tasa de aprendizaje y su gradiente, y esa tasa de aprendizaje es un hiperparámetro que controla el tamaño del paso, pues valores excesivos hacen oscilar la pérdida y valores pequeños ralentizan la convergencia; la inicialización de los pesos también condiciona el aprendizaje, por lo que para capas con ReLU se emplea la inicialización de He, que preserva la varianza de la señal a través de la red (He et al., 2015).

Clasificación Binaria y Clasificación Multiclase

Ambos problemas comparten estructura pero exigen configuraciones distintas, ya que en la clasificación binaria basta una neurona de salida con sigmoide cuya probabilidad se compara contra un umbral de 0.5, mientras en la clasificación multiclase se requiere una neurona por clase con

activación softmax y la decisión corresponde a la posición del valor mayor, obtenida con la operación argmax; la pérdida cambia en consecuencia y la evaluación debe ampliarse, porque además de la exactitud global la matriz de confusión revela qué clases se confunden entre sí y el reporte por clase precisión y recuerdo (Gulli et al., 2023).

TensorFlow, Keras y PyTorch

TensorFlow es una biblioteca de cómputo numérico basada en tensores y grafos con diferenciación automática, Keras es su interfaz de alto nivel para construir y entrenar modelos mediante capas declarativas y el método de ajuste fit, que encapsula forward propagation, pérdida, backpropagation y actualización (Gulli et al., 2023), y PyTorch, en cambio, expone el ciclo de entrenamiento de manera explícita, de modo que el programador limpia los gradientes acumulados, propaga hacia adelante, calcula la pérdida, ejecuta backward para obtener los gradientes y aplica el paso del optimizador (Stevens et al., 2020); lejos de ser matemáticas distintas, ambos frameworks automatizan los mismos fundamentos que en este trabajo se implementaron primero a mano, diferencia que se discute en la sección de resultados.

Metodología

Entorno de Trabajo

Las seis prácticas se desarrollaron como notebooks de Google Colaboratory y se ejecutaron de extremo a extremo con semillas fijas en 42 para garantizar reproducibilidad, la Tabla 1 resume los componentes del entorno, y todas las salidas reportadas en este documento provienen de esa ejecución real y se conservan, junto con las gráficas, en el repositorio público de la actividad (Salazar Cardenas, 2026).

Tabla 1

Componentes y Versiones del Entorno de Trabajo

Componente	**Versión o valor**
Lenguaje de programación	Python 3.11
Cálculo numérico	NumPy 2.4.6
Preprocesamiento y métricas	scikit-learn 1.9.1
Visualización	Matplotlib 3.11.2
Framework 1	TensorFlow 2.21.0 con Keras 3.15.1 (backend TensorFlow)
Framework 2	PyTorch 2.14.1 con torchvision 0.29.1
Dataset	MNIST: 60 000 imágenes de entrenamiento y 10 000 de prueba, 28 × 28 píxeles
Dispositivo de ejecución	CPU de dos núcleos con aproximadamente 1 GB de RAM
Semilla de reproducibilidad	42 en todas las prácticas

Nota. Los notebooks incluyen metadata de Colaboratory y pueden ejecutarse en ese entorno sin modificaciones.

Datos y Preprocesamiento

Se utilizaron dos conjuntos de datos, el primero sintético y didáctico de frutas con tres características —peso en gramos, longitud en centímetros y nivel de amarillo entre cero y uno— y dos o tres clases según la práctica, y el segundo MNIST, base de referencia para clasificación multiclase de imágenes (LeCun et al., 2010); en ambos casos los datos se dividieron en conjuntos de entrenamiento y prueba mediante partición estratificada con semilla fija, de modo que las muestras de prueba nunca participaron en el ajuste de parámetros, las características de las frutas se estandarizaron con media cero y desviación uno ajustando el escalador únicamente con el entrenamiento, y los píxeles de MNIST se normalizaron al intervalo entre

cero y uno dividiendo entre 255, con cada imagen de 28 por 28 aplanada en un vector de 784 características.

Procedimiento por Práctica

La Tabla 2 describe la ruta de trabajo, y en cada práctica se inspeccionaron los datos antes de modelar, se implementó el modelo, se entrenó registrando la pérdida por época y se evaluó con pérdida, exactitud, matriz de confusión y análisis de errores concretos; la implementación manual con NumPy precede siempre a la automatización con frameworks, de manera que cada operación que Keras o PyTorch encapsulan quedara primero comprendida y verificada.

Tabla 2

Ruta de Trabajo de las Seis Prácticas

Práctica	Modelo y tecnología	Problema	Aprendizaje central
1	Perceptrón con escalón (NumPy)	Manzana frente a banano	Regla de aprendizaje y convergencia
2	Red de una capa y red multicapa (NumPy)	Manzana frente a banano	Sigmoide, ReLU, backpropagation manual y desvanecimiento del gradiente
3	Red 3-4-1 (TensorFlow y Keras)	Manzana frente a banano	Automatización del entrenamiento con fit
4	Red 3-6-3 con softmax (TensorFlow y Keras)	Manzana, banano y naranja	Clasificación multiclase y matriz de confusión
5	Red 784-128-10 (TensorFlow y Keras)	Dígitos 0 a 9 de MNIST	Multiclase sobre imágenes sin convoluciones
6	Red 784-128-10 (PyTorch)	Dígitos 0 a 9 de MNIST	Ciclo explícito de entrenamiento y gradientes inspeccionables

Nota. Ninguna práctica emplea capas convolucionales, condición verificada programáticamente en los notebooks 5 y 6.

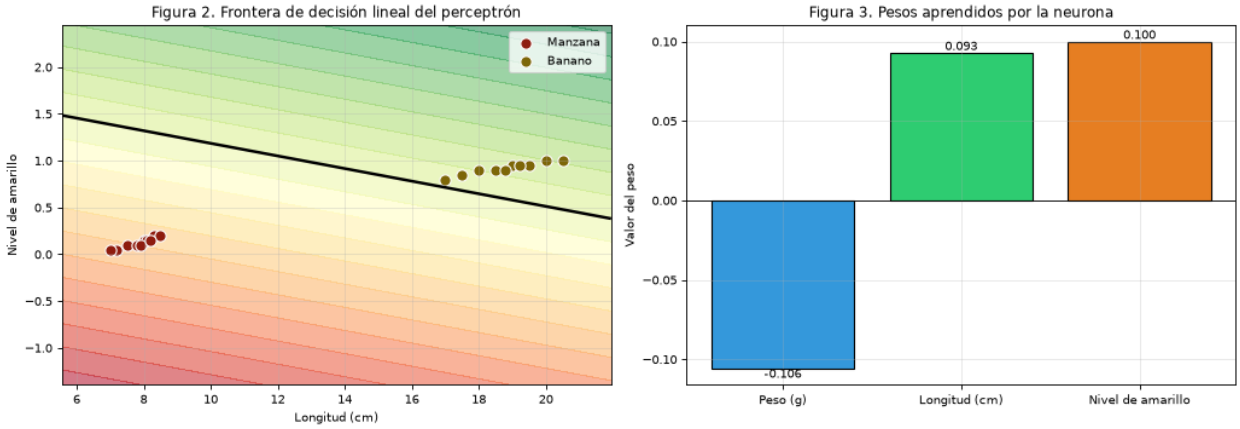
Resultados

Práctica 1: Perceptrón Implementado con NumPy

El perceptrón convergió en dos épocas con una única actualización efectiva de pesos, señal de que el conjunto de frutas es linealmente separable, y clasificó correctamente las cuatro frutas de prueba, mientras la Figura 1 muestra la frontera de decisión lineal obtenida y los pesos aprendidos; como se anticipó en el marco conceptual, el modelo entrega una clase discreta y no una probabilidad, y su superficie de decisión es necesariamente un hiperplano, limitaciones ambas que motivan la práctica siguiente.

Figura 1

Frontera de decisión y pesos aprendidos por el perceptrón



Nota. Elaboración propia a partir de la ejecución del Notebook 01 (Salazar Cardenas, 2026). La línea negra corresponde al umbral de decisión.

Práctica 2: Red de una Capa y Red Multicapa con Backpropagation

La red de una capa con sigmoide redujo su pérdida de 0.6929 a 0.0018 en 2000 épocas, una disminución del 99.73 %, y la red multicapa 3-4-1 con ReLU alcanzó 0.0079 en 3000 épocas, clasificando ambas correctamente las frutas de prueba, y la verificación crucial del cálculo de gradientes consiste en que la pérdida disminuye tras un único paso de

actualización, de 0.692932 a 0.624232, lo que confirma que las derivadas implementadas a mano son correctas; la Tabla 3 consolida los tres modelos manuales.

Tabla 3

Resultados de los Modelos Implementados con NumPy y Backpropagation Manual

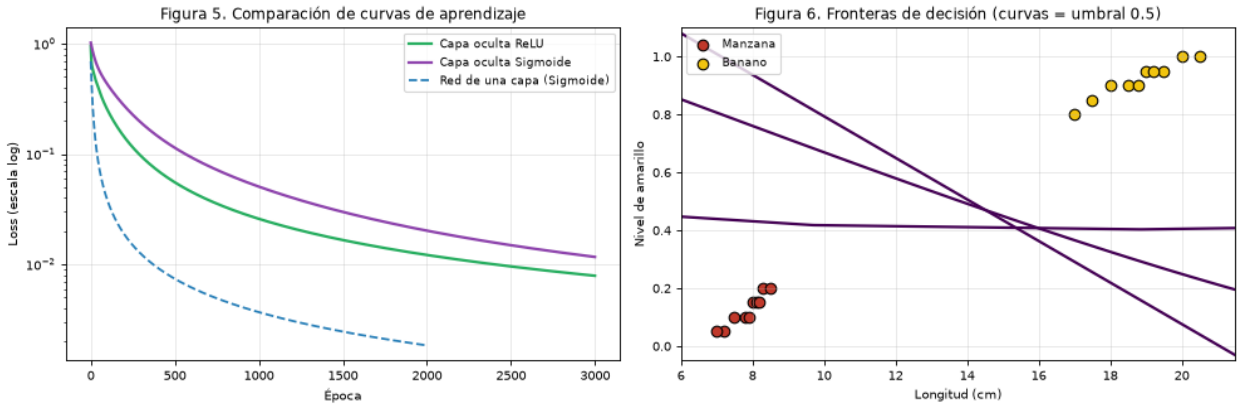
Modelo	Pérdida inicial	Pérdida final	Pérdida en prueba	Exactitud en prueba
Una capa con sigmoide (2000 épocas)	0.692932	0.001846	0.002713	1.0000
Multicapa con ReLU (3000 épocas)	0.952865	0.007896	0.011104	1.0000
Multicapa con sigmoide oculta (3000 épocas)	1.022297	0.011692	0.013006	1.0000

Nota. Mismos datos, misma partición y misma semilla en los tres modelos. La exactitud sobre cuatro frutas de prueba debe leerse como verificación de funcionamiento y no como medida de generalización.

El experimento controlado entre activaciones aporta la evidencia más relevante de esta práctica, pues con idéntica arquitectura, datos y semilla la norma del gradiente de la primera capa al inicio del entrenamiento fue de 1.158135 con ReLU frente a 0.437588 con sigmoide, una razón de 2.65 veces; el resultado ilustra el desvanecimiento del gradiente que produce una activación cuya derivada nunca supera 0.25 y justifica el uso de ReLU en capas ocultas (He et al., 2015), y la Figura 2 compara las curvas de aprendizaje y las fronteras de decisión de los tres modelos.

Figura 2

Comparación de curvas de aprendizaje y fronteras de decisión entre activaciones



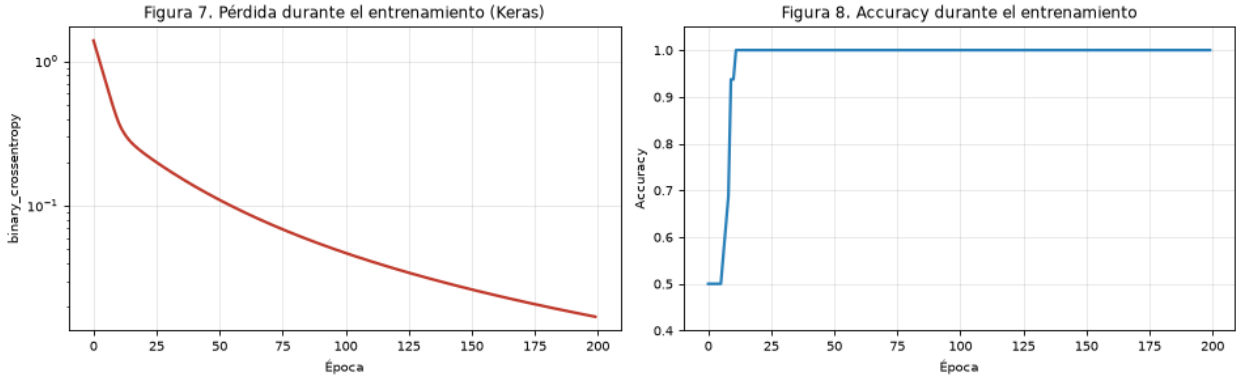
Nota. Elaboración propia a partir de la ejecución del Notebook 02. El panel izquierdo usa escala logarítmica en la pérdida.

Práctica 3: Clasificación Binaria con TensorFlow y Keras

El modelo Sequential de Keras con la misma arquitectura 3-4-1 declaró 21 parámetros entrenables, exactamente los mismos que la implementación manual, y alcanzó una pérdida de prueba de 0.017033 con exactitud de 1.0000 tras 200 épocas con el optimizador Adam, mientras la probabilidad asignada a la fruta nueva de prueba fue de 0.999793 para la clase banano, coincidente con la predicción del perceptrón y de la red manual; la Figura 3 muestra las curvas de pérdida y exactitud del entrenamiento.

Figura 3

Pérdida y exactitud durante el entrenamiento del modelo binario en Keras



Nota. Elaboración propia a partir de la ejecución del Notebook 03. El panel izquierdo usa escala logarítmica.

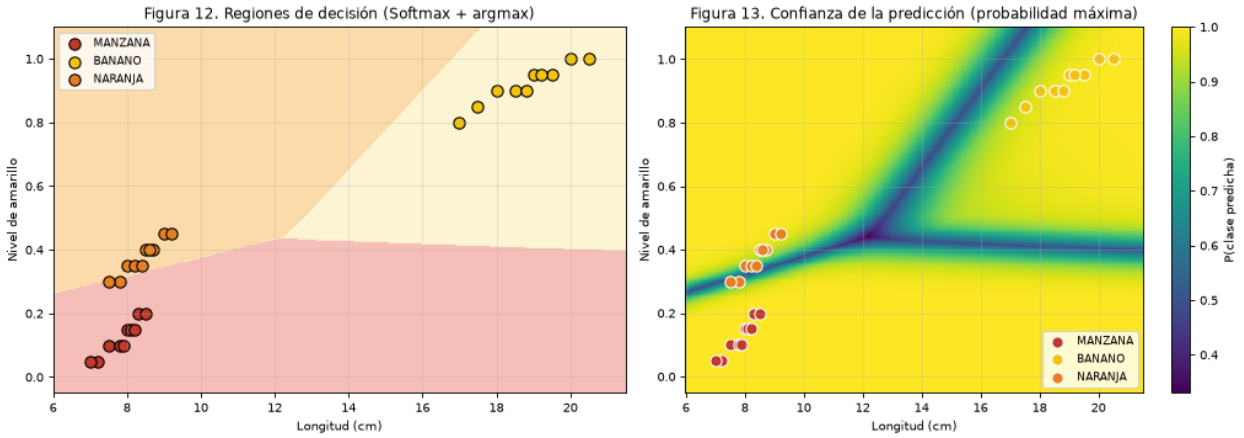
La inspección de los pesos con get_weights confirmó el comportamiento esperado de la capa ReLU: para las manzanas de prueba las cuatro neuronas ocultas permanecen en cero, mientras que para los bananos se activan con valores de 5.07 y 3.11, es decir, la red acumula evidencia a favor de la clase positiva solo cuando las características la sustentan.

Práctica 4: Clasificación Multiclase con TensorFlow y Keras

Al incorporar la tercera clase fueron necesarios tres cambios coordinados, a saber, tres neuronas de salida, activación softmax y pérdida de entropía cruzada categórica dispersa, y el modelo 3-6-3 redujo su pérdida de 1.4699 a 0.0056 en 300 épocas, obtuvo exactitud de 1.0000 sobre seis frutas de prueba y produjo una matriz de confusión diagonal; la probabilidad máxima promedio sobre la malla de decisión fue de 0.9360, con apenas 4.20 % del área por debajo de 0.60, lo que indica fronteras bien definidas, y la Figura 4 presenta las regiones de decisión y la confianza del modelo.

Figura 4

Regiones de decisión y confianza de la predicción en el problema multiclase de frutas



Nota. Elaboración propia a partir de la ejecución del Notebook 04. El panel derecho muestra la probabilidad máxima asignada en cada punto del espacio de características.

Práctica 5: MNIST con TensorFlow y Keras

Sobre MNIST se entrenó un perceptrón multicapa 784-128-10 con 101 770 parámetros, cinco épocas, lotes de 128 imágenes y validación del 10 %, y la Tabla 4 resume los resultados, pues la exactitud sobre las 10 000 imágenes de prueba fue de 0.9735, con 265 errores y una pérdida de 0.089279; la confianza promedio del modelo fue de 0.9764 en los aciertos y de 0.6941 en los errores, de modo que la probabilidad máxima constituye una señal útil para decidir cuándo derivar un caso a revisión humana.

Tabla 4

Resultados de la Clasificación de Dígitos MNIST sin Redes Convolucionales

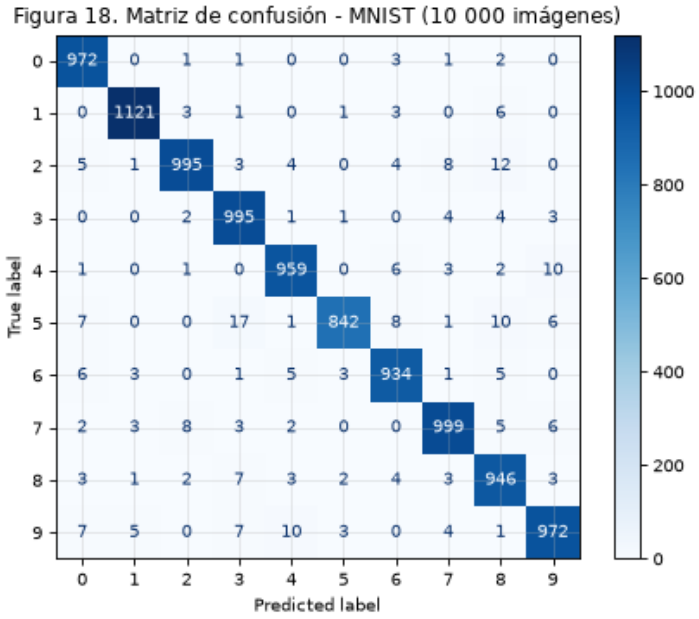
Indicador	**TensorFlow y Keras**	**PyTorch**
Arquitectura	784-128-10 (ReLU y softmax)	784-128-10 (ReLU y logits)
Parámetros entrenables	101 770	101 770
Épocas y tamaño de lote	5 épocas, lote de 128	3 épocas, lote de 64
Pérdida de prueba	0.089279	0.1061 en entrenamiento
Exactitud de prueba	0.9735 (9 735 de 10 000)	0.9690 (9 690 de 10 000)
Confusiones más frecuentes	5→3, 2→8, 9→4, 5→8, 4→9	8→3, 9→7, 9→4, 8→7, 2→7
Mejor y peor recuerdo por clase	Dígito 0 (0.9918) y dígito 5 (0.9439)	Dígito 0 (0.9867) y dígito 8 (0.9302)
Tiempo de entrenamiento	8.7 s en CPU	15.2 s en CPU

Nota. Ambos modelos comparten arquitectura y dataset con la partición oficial. Las diferencias de exactitud se explican por el número de épocas y el tamaño de lote, no por el framework.

El análisis de errores muestra que las confusiones se concentran en pares morfológicamente similares, como 5 y 3 o 4 y 9, patrón coherente con un modelo que observa intensidades de píxeles aplanadas sin información explícita de forma local, que es precisamente la ventaja que aportaría una capa convolucional excluida por la consigna; la Figura 5 exhibe la matriz de confusión y la Figura 6 algunos dígitos erróneos, y un experimento complementario sobre la capacidad de la capa oculta mostró que pasar de 32 a 128 neuronas eleva la exactitud de 0.9124 a 0.9311, mientras duplicar nuevamente a 256 neuronas apenas aporta 0.9328, de modo que más parámetros no garantizan mejor modelo.

Figura 5

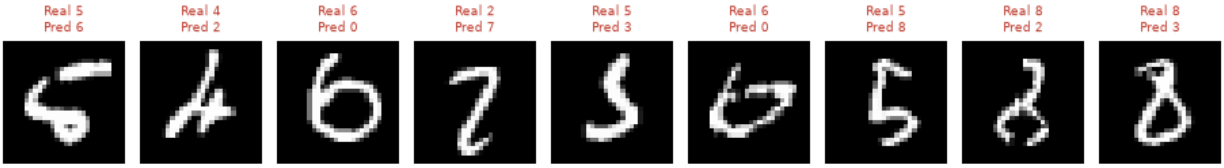
Matriz de confusión del clasificador de dígitos entrenado con Keras



Nota. Elaboración propia a partir de la ejecución del Notebook 05 sobre las 10 000 imágenes de prueba.

Figura 6

Dígitos clasificados incorrectamente por el modelo de Keras



Nota. Elaboración propia a partir de la ejecución del Notebook 05. Cada panel indica la etiqueta real y la clase predicha.

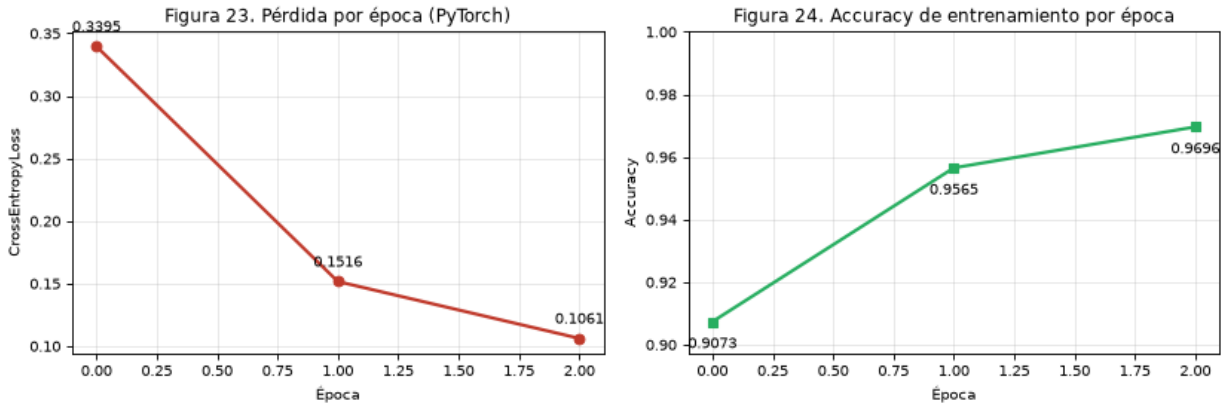
Práctica 6: MNIST con PyTorch

La misma arquitectura se implementó como una clase que hereda de nn.Module y se entrenó con el ciclo explícito de cinco pasos, esto es, limpiar gradientes, propagar hacia adelante, calcular la pérdida, propagar hacia atrás y aplicar el paso del optimizador, y la pérdida promedio por época

descendió de 0.3395 a 0.1061 con una exactitud de prueba de 0.9690; la inspección del atributo grad tras backward confirmó que ninguna norma es nula —0.534879 en el peso de la capa oculta y 0.718763 en el de salida—, evidencia directa de que la señal de error alcanzó todas las capas, y la Figura 7 muestra las curvas de pérdida y exactitud mientras la Figura 8 presenta la matriz de confusión correspondiente.

Figura 7

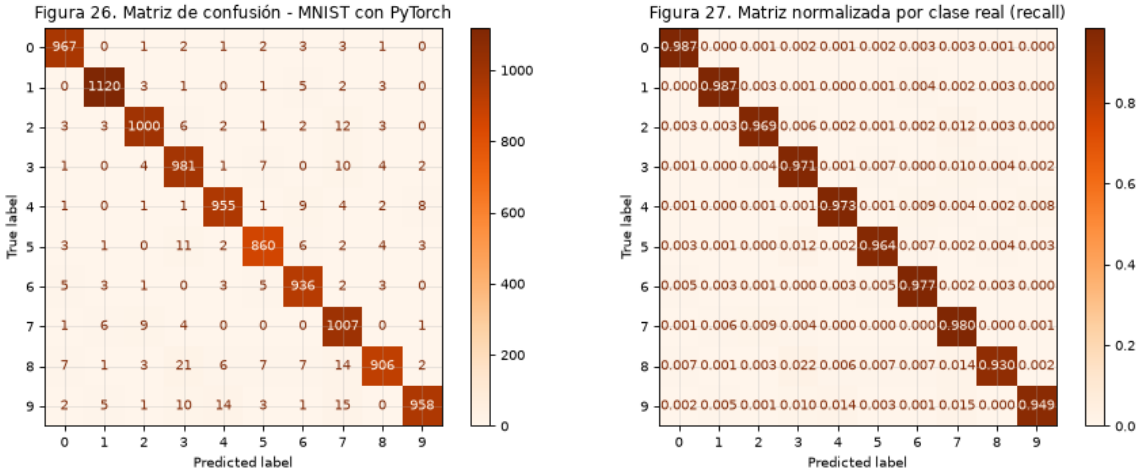
Pérdida y exactitud de entrenamiento por época en PyTorch



Nota. Elaboración propia a partir de la ejecución del Notebook 06.

Figura 8

Matriz de confusión del clasificador de dígitos implementado en PyTorch



Nota. Elaboración propia a partir de la ejecución del Notebook 06. El panel derecho normaliza por clase real.

Comparación de Enfoques y Análisis General

La Tabla 5 compara los tres niveles de implementación utilizados, y la diferencia esencial no es matemática sino de explicitud, ya que NumPy obliga a escribir cada derivada, Keras encapsula el ciclo completo en el método de ajuste y PyTorch lo expone como un bucle de cinco instrucciones que el programador controla; quien domina forward propagation, pérdida, backpropagation y actualización puede migrar entre las tres formas sin reaprender la teoría y, lo más importante, puede diagnosticar por qué un modelo no aprende.

Tabla 5

Comparación de los Tres Niveles de Implementación

Acción	**NumPy manual**	**TensorFlow y Keras**	**PyTorch**
Creación de capas	Matrices de pesos y sesgos	Capas densas declarativas	Módulos lineales
Activaciones	Funciones propias	Argumento de la capa	Módulos de activación
Pérdida	Función propia	Argumento de compilación	Módulo de criterio
Backpropagation	Derivadas escritas a mano	Automática dentro del ajuste	Método backward
Actualización de pesos	Resta manual del gradiente	Automática por el optimizador	Método step del optimizador
Escalabilidad	Inviable en modelos grandes	Diseñada para GPU y modelos grandes	Diseñada para GPU y modelos grandes
Valor principal	Comprensión y verificación	Productividad y despliegue	Control y depuración

Nota. Los tres niveles resuelven los mismos problemas con idéntico fundamento matemático.

Tres lecturas críticas cierran la sección: primera, las exactitudes de 1.0000 obtenidas con frutas son verificaciones de correcto funcionamiento sobre conjuntos de cuatro y seis muestras y no medidas de generalización, pues la evidencia sólida corresponde a MNIST, evaluado sobre 10 000 imágenes no vistas; segunda, la pérdida y la confianza aportan más información que la exactitud, porque permiten identificar cuándo el modelo duda; y tercera, los errores no son aleatorios sino estructurales, concentrados en pares de dígitos semejantes, lo que delimita con precisión qué ganaría una arquitectura convolucional que la consigna excluyó.

Conclusiones

Una red neuronal aprende cuando el error de sus predicciones se utiliza para modificar sus parámetros, y el ciclo completo de forward propagation, cálculo de pérdida, backpropagation y actualización se implementó manualmente y se verificó observando que la pérdida disminuye tras cada paso de gradiente, confirmación empírica de que los gradientes están bien calculados.

Backpropagation no es un algoritmo de aprendizaje sino el cálculo eficiente de los gradientes mediante la regla de la cadena, de modo que el optimizador es quien aprende al consumirlos, y distinguir ambos roles resulta decisivo para diagnosticar modelos que no convergen.

La elección de la función de activación depende del lugar que ocupa en la red, pues ReLU conviene en capas ocultas por su gradiente unitario, sigmoide en la salida binaria por su interpretabilidad probabilística y softmax en la salida multiclase por convertir puntuaciones en una distribución normalizada; el experimento controlado cuantificó la consecuencia de elegir mal, ya que la norma del gradiente fue 2.65 veces mayor con ReLU que con sigmoide en la misma arquitectura.

La implementación manual con NumPy es lo que permite comprender qué automatizan los frameworks, puesto que los 21 parámetros gestionados a mano coinciden con los que declara el resumen del modelo en Keras y los 101 770 parámetros de MNIST son idénticos en Keras y en PyTorch; el método de ajuste oculta el ciclo, mientras los métodos backward y step lo exponen.

El tránsito de binario a multiclase exige tres cambios coordinados, a saber, número de neuronas de salida, activación softmax y pérdida de

entropía cruzada multiclase, de modo que alterar solo uno de ellos produce errores de forma o entrenamientos que no convergen.

La progresión de frutas a MNIST demuestra que los mismos principios escalan, pues se pasó de tres características y veinte muestras a 784 características y sesenta mil muestras, y de dos clases a diez, sin modificar la lógica del modelo; el resultado obtenido sin redes convolucionales, 0.9735 con Keras y 0.9690 con PyTorch sobre diez mil imágenes de prueba, es coherente con lo esperado para un perceptrón multicapa de una capa oculta.

La evaluación profesional de un clasificador va más allá de la exactitud, ya que incluye pérdida, matriz de confusión, reporte por clase, confianza de las predicciones y análisis de errores concretos, y en MNIST la confianza promedio fue de 0.9764 en aciertos y 0.6941 en errores, con las confusiones concentradas en pares morfológicamente similares.

Finalmente, la reproducibilidad forma parte del rigor técnico, pues semillas fijas, versiones documentadas, salidas conservadas y código publicado en un repositorio público permiten que un tercero verifique los resultados (Salazar Cardenas, 2026), y un resultado que no puede reproducirse no constituye evidencia.

Referencias Bibliográficas

GitHub. (2026). GitHub [Plataforma de alojamiento de código]. https://github.com/

Google. (2026). Google Colaboratory [Entorno de notebooks en la nube]. Google Research. https://colab.research.google.com/

Gulli, A., Kapoor, A., Pal, S., & Ciraci, S. (2023). Deep learning with TensorFlow and Keras (3.ª ed.). Packt Publishing. https://download.packt.com/free-ebook/9781803232911

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. En Proceedings of the IEEE International Conference on Computer Vision (pp. 1026-1034). IEEE.

LeCun, Y., Cortes, C., & Burges, C. J. C. (2010). MNIST handwritten digit database. AT&T Labs Research. http://yann.lecun.com/exdb/mnist/

Packt Publishing. (2023). Deep Learning with TensorFlow and Keras - 3rd edition repository [Código fuente]. GitHub. https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386-408.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323(6088), 533-536.

Salazar Cardenas, Y. J. N. (2026). Redes Neuronales II [Código fuente]. GitHub. https://github.com/Yohann0011/Redes-Neuronales-II

Stevens, E., Antiga, L., & Viehmann, T. (2020). Deep learning with PyTorch. Manning Publications. https://isip.piconepress.com/courses/temple/ece_4822/resources/books/Deep-Learning-with-PyTorch.pdf

Trask, A. (2019). Grokking deep learning. Manning Publications. https://edu.anarcho-copy.org/Algorithm/grokking-deep-learning.pdf

Universidad de Cundinamarca. (2026). Guía de trabajo y laboratorio: Redes Neuronales II [Material del curso Deep Learning - Conceptos]. Especialización en Inteligencia Artificial.

Apéndice A

Código Fuente del Notebook 01: Perceptrón con NumPy

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import random

random.seed(42)
np.random.seed(42)
print("NumPy:", np.__version__)
print("Herramientas cargadas correctamente")

X = np.array([
    # MANZANAS  (clase 0): [peso g, longitud cm, nivel de amarillo 0-1]
    [150, 7.5, 0.10], [160, 8.0, 0.15], [145, 7.2, 0.05],
    [170, 8.3, 0.20], [155, 7.8, 0.10], [165, 8.1, 0.15],
    [140, 7.0, 0.05], [175, 8.5, 0.20], [158, 7.9, 0.10],
    [162, 8.2, 0.15],

    # BANANOS   (clase 1)
    [120, 18.0, 0.90], [125, 19.0, 0.95], [115, 17.5, 0.85],
    [130, 20.0, 1.00], [118, 18.5, 0.90], [123, 19.5, 0.95],
    [110, 17.0, 0.80], [128, 20.5, 1.00], [121, 18.8, 0.90],
    [126, 19.2, 0.95]
], dtype=float)

y = np.array([0] * 10 + [1] * 10)

print("Matriz de características X:", X.shape, "-> (20 frutas, 3 características)")
print("Vector de etiquetas   y:", y.shape, "-> (0 = manzana, 1 = banano)")
print()
print("Primeras 3 manzanas:\n", X[:3])
print("Primeros 3 bananos:\n", X[10:13])


from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 80 % entrenamiento / 20 % prueba, conservando la proporción de clases
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# El escalado se AJUSTA solo con entrenamiento y se APLICA a prueba
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
print("X_train:", X_train.shape, "| X_test:", X_test.shape)
print("y_train:", y_train.shape, "| y_test:", y_test.shape)
print("Distribución de clases en prueba:", np.bincount(y_test))
print()
print("Media de cada característica en entrenamiento (debe ser ~0):")
print(np.round(X_train.mean(axis=0), 6))
print("Desviación estándar (debe ser ~1):")
print(np.round(X_train.std(axis=0), 6))


fig, axes = plt.subplots(1, 3, figsize=(13, 3.6))
pares = [(0, 1, "Peso (g)", "Longitud (cm)"),
         (1, 2, "Longitud (cm)", "Nivel de amarillo"),
         (0, 2, "Peso (g)", "Nivel de amarillo")]
colores = ["#c0392b", "#f1c40f"]

for ax, (i, j, xl, yl) in zip(axes, pares):
    for clase, nombre in [(0, "Manzana"), (1, "Banano")]:
        m = y == clase
        ax.scatter(X[m, i], X[m, j], c=colores[clase], label=nombre,
                   edgecolors="k", s=55, alpha=0.9)
    ax.set_xlabel(xl); ax.set_ylabel(yl); ax.legend(fontsize=8)
    ax.set_title(f"{xl} vs {yl}", fontsize=10)

fig.suptitle("Figura 1. Distribución de las dos clases en el espacio de características", y=1.02)
plt.tight_layout(); plt.savefig("fig01_datos_frutas.png", bbox_inches="tight"); plt.show()

def step(z):
    """Función de activación escalón (umbral en 0)."""
    return (z >= 0).astype(int)


def predecir(X_datos, w, b):
    """Forward propagation de una sola neurona: z = X·w + b  ->  y_hat = step(z)."""
    z = X_datos @ w + b
    return step(z)


# --- Inicialización de parámetros ---
np.random.seed(42)
w = np.random.randn(3) * 0.01     # pesos pequeños aleatorios
b = 0.0                           # bias inicial

print("Pesos iniciales w =", np.round(w, 6))
print("Bias inicial    b =", round(b, 6))
print()
print("Predicciones iniciales (sin entrenar):", predecir(X_train, w, b))
print("Etiquetas reales                     :", y_train)
print("Aciertos iniciales:", int((predecir(X_train, w, b) == y_train).sum()), "de", len(y_train))

np.random.seed(42)
w = np.random.randn(3) * 0.01
b = 0.0
eta = 0.1

print("=== ITERACIÓN 1 (paso a paso) ===")
print(f"{'#':>2} {'z':>9} {'y_hat':>5} {'y':>3} {'error':>6}   acción")
for k in range(len(X_train)):
    x_k, y_k = X_train[k], y_train[k]
    z_k = float(x_k @ w + b)
    y_hat = int(z_k >= 0)
    error = y_k - y_hat
    accion = "sin cambio (acierto)" if error == 0 else "se actualizan w y b"
    print(f"{k:>2} {z_k:>9.4f} {y_hat:>5} {y_k:>3} {error:>6}   {accion}")
    w = w + eta * error * x_k
```

```
    b = b + eta * error

print()
print("Pesos tras la iteración 1:", np.round(w, 6))
print("Bias  tras la iteración 1:", round(b, 6))

np.random.seed(42)
w = np.random.randn(3) * 0.01
b = 0.0
eta = 0.1
max_epochs = 100

historial_accuracy = []
epoch_convergencia = None

for epoch in range(max_epochs):
    actualizaciones = 0
    for k in range(len(X_train)):
        x_k, y_k = X_train[k], y_train[k]
        y_hat = int((x_k @ w + b) >= 0)
        error = y_k - y_hat
        if error != 0:
            w = w + eta * error * x_k
            b = b + eta * error
            actualizaciones += 1

    acc = float((predecir(X_train, w, b) == y_train).mean())
    historial_accuracy.append(acc)

    if epoch % 10 == 0 or actualizaciones == 0:
        print(f"Época {epoch:>3} | actualizaciones: {actualizaciones:>2} | accuracy entrenamiento:
{acc:.4f}")

    if actualizaciones == 0 and epoch_convergencia is None:
        epoch_convergencia = epoch
        print(f"\n>>> Convergió en la época {epoch}: ninguna predicción fue incorrecta.")
        break

print()
print("Pesos finales   w =", np.round(w, 4))
print("Bias final      b =", round(b, 4))
print("Épocas usadas     =", epoch + 1)

y_pred_test = predecir(X_test, w, b)
acc_test = accuracy_score(y_test, y_pred_test)

print("=== PREDICCIONES EN EL CONJUNTO DE PRUEBA ===")
print(f"{'Fruta':>6} {'Prob. z':>9} {'Predicción':>12} {'Real':>12} {'Resultado':>10}")
for i in range(len(X_test)):
    z_i = float(X_test[i] @ w + b)
    pred = "BANANO" if y_pred_test[i] == 1 else "MANZANA"
    real = "BANANO" if y_test[i] == 1 else "MANZANA"
    ok = "ACIERTO" if pred == real else "ERROR"
    print(f"{i+1:>6} {z_i:>9.4f} {pred:>12} {real:>12} {ok:>10}")

print()
print("Accuracy en prueba:", round(acc_test, 4))
print()
print("Matriz de confusión (filas = reales, columnas = predichas):")
print(confusion_matrix(y_test, y_pred_test))
print()
print(classification_report(y_test, y_pred_test, target_names=["Manzana", "Banano"],
zero_division=0))

# Deshacer el escalado para graficar en unidades originales
```

```
media, desv = scaler.mean_, scaler.scale_

i_feat, j_feat = 1, 2      # longitud (cm) vs nivel de amarillo
rangos = np.array([X[:, i_feat].min(), X[:, i_feat].max(),
                   X[:, j_feat].min(), X[:, j_feat].max()])
margen = 0.1 * (rangos[1] - rangos[0] + rangos[3] - rangos[2])
g1 = np.linspace(rangos[0] - margen, rangos[1] + margen, 300)
g2 = np.linspace(rangos[2] - margen, rangos[3] + margen, 300)
G1, G2 = np.meshgrid(g1, g2)

# Se reconstruye el vector de 3 características usando la media para la tercera
puntos = np.column_stack([
    np.full(G1.size, media[0]),
    G1.ravel(),
    G2.ravel()
])
puntos_escalados = (puntos - media) / desv
Z_grid = (puntos_escalados @ w + b).reshape(G1.shape)

fig, axes = plt.subplots(1, 2, figsize=(13, 4.6))

# (a) Frontera de decisión
ax = axes[0]
ax.contourf(G1, G2, Z_grid, levels=20, cmap="RdYlGn", alpha=0.55)
ax.contour(G1, G2, Z_grid, levels=[0], colors="k", linewidths=2.2)
for clase, nombre, c in [(0, "Manzana", "#8e1b0e"), (1, "Banano", "#7d6608")]:
    m = y == clase
    ax.scatter(X[m, i_feat], X[m, j_feat], c=c, edgecolors="w", s=70, label=nombre)
ax.set_xlabel("Longitud (cm)"); ax.set_ylabel("Nivel de amarillo")
ax.set_title("Figura 2. Frontera de decisión lineal del perceptrón")
ax.legend()

# (b) Pesos aprendidos
ax = axes[1]
etiquetas = ["Peso (g)", "Longitud (cm)", "Nivel de amarillo"]
barras = ax.bar(etiquetas, w, color=["#3498db", "#2ecc71", "#e67e22"], edgecolor="k")
ax.axhline(0, color="k", lw=0.8)
ax.set_ylabel("Valor del peso")
ax.set_title("Figura 3. Pesos aprendidos por la neurona")
for barra, valor in zip(barras, w):
    ax.text(barra.get_x() + barra.get_width() / 2, valor,
            f"{valor:.3f}", ha="center",
            va="bottom" if valor >= 0 else "top", fontsize=9)
ax.grid(axis="x", alpha=0.3)

plt.tight_layout(); plt.savefig("fig02_perceptron_frontera.png", bbox_inches="tight"); plt.show()

print("Lectura de los pesos:")
print(" · La característica con mayor |peso| domina la decisión.")
print(f" · |w| = {np.round(np.abs(w), 4)}")

fruta_nueva = np.array([[124.0, 19.0, 0.94]])      # liviana, alargada, muy amarilla
fruta_escalada = scaler.transform(fruta_nueva)

z_nueva = float((fruta_escalada @ w + b)[0])
pred_nueva = int(z_nueva >= 0)
nombre_pred = "BANANO" if pred_nueva == 1 else "MANZANA"

print("Fruta nueva (peso, longitud, amarillo):", fruta_nueva[0])
print("Valores escalados                     :", np.round(fruta_escalada[0], 4))
print("z = w·x + b                           :", round(z_nueva, 4))
print("step(z)                               :", pred_nueva)
print("La red piensa que es                  :", nombre_pred)
```

Nota. El código se presenta a un espacio por tratarse de material informático. La ejecución completa con sus salidas y gráficas se conserva en los notebooks del repositorio (Salazar Cardenas, 2026) y en las evidencias que lo acompañan.

Apéndice B

Código Fuente del Notebook 02: Red de una Capa y Red Multicapa con Backpropagation

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import random
random.seed(42)
np.random.seed(42)
print("NumPy:", np.__version__, "| entorno listo")

X = np.array([
    # MANZANAS  (clase 0): [peso g, longitud cm, nivel de amarillo 0-1]
    [150, 7.5, 0.10], [160, 8.0, 0.15], [145, 7.2, 0.05],
    [170, 8.3, 0.20], [155, 7.8, 0.10], [165, 8.1, 0.15],
    [140, 7.0, 0.05], [175, 8.5, 0.20], [158, 7.9, 0.10],
    [162, 8.2, 0.15],

    # BANANOS   (clase 1)
    [120, 18.0, 0.90], [125, 19.0, 0.95], [115, 17.5, 0.85],
    [130, 20.0, 1.00], [118, 18.5, 0.90], [123, 19.5, 0.95],
    [110, 17.0, 0.80], [128, 20.5, 1.00], [121, 18.8, 0.90],
    [126, 19.2, 0.95]
], dtype=float)

y = np.array([0] * 10 + [1] * 10)

print("Matriz de características X:", X.shape, "-> (20 frutas, 3 características)")
print("Vector de etiquetas   y:", y.shape, "-> (0 = manzana, 1 = banano)")
print()
print("Primeras 3 manzanas:\n", X[:3])
print("Primeros 3 bananos:\n", X[10:13])


from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 80 % entrenamiento / 20 % prueba, conservando la proporción de clases
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# El escalado se AJUSTA solo con entrenamiento y se APLICA a prueba
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
print("X_train:", X_train.shape, "| X_test:", X_test.shape)
print("y_train:", y_train.shape, "| y_test:", y_test.shape)
print("Distribución de clases en prueba:", np.bincount(y_test))
print()
print("Media de cada característica en entrenamiento (debe ser ~0):")
print(np.round(X_train.mean(axis=0), 6))
print("Desviación estándar (debe ser ~1):")
print(np.round(X_train.std(axis=0), 6))


def sigmoid(z):
    """Función de activación Sigmoide: comprime z al intervalo (0, 1)."""
    return 1.0 / (1.0 + np.exp(-z))


def relu(z):
    """Función de activación ReLU: los negativos se anulan, los positivos pasan."""
    return np.maximum(0.0, z)


def derivada_relu(z):
    """Derivada de ReLU: 1 si z > 0, 0 en caso contrario."""
    return (z > 0).astype(float)


def calcular_error(y_real, y_predicha):
    """Binary Cross-Entropy promedio."""
    epsilon = 1e-8
    return -np.mean(
        y_real * np.log(y_predicha + epsilon)
        + (1 - y_real) * np.log(1 - y_predicha + epsilon)
    )


# --- Verificación numérica de las activaciones ---
z_prueba = np.array([-3.0, -1.0, 0.0, 1.0, 3.0])
print("z            :", z_prueba)
print("sigmoid(z)   :", np.round(sigmoid(z_prueba), 4))
print("relu(z)      :", relu(z_prueba))
print("relu'(z)     :", derivada_relu(z_prueba))
print()
print("Error si predice 0.5 para y=1 :", round(float(calcular_error(np.array([[1.0]]),
np.array([[0.5]]))), 4))
print("Error si predice 0.99 para y=1:", round(float(calcular_error(np.array([[1.0]]),
np.array([[0.99]]))), 4))
print("Error si predice 0.01 para y=1:", round(float(calcular_error(np.array([[1.0]]),
np.array([[0.01]]))), 4))
print("-> La pérdida crece cuando la red está segura y se equivoca: por eso penaliza más los
errores graves.")

np.random.seed(42)
W = np.random.randn(3, 1) * 0.01
b = 0.0

Z = X_train.reshape(-1, 3) @ W + b          # combinación lineal
A = sigmoid(Z)                              # activación

y_col = y_train.reshape(-1, 1)              # etiquetas en forma de columna

print("Pesos iniciales W =", np.round(W.ravel(), 6), "| b =", round(b, 6))
print()
print("Primeras 5 predicciones (probabilidades):", np.round(A.ravel()[:5], 4))
print("Primeras 5 etiquetas reales             :", y_train[:5])
print()
```

```
print("Error inicial (BCE):", round(float(calcular_error(y_col, A)), 4))
print("-> Un valor cercano a ln(2) = 0.6931 indica que la red predice ~0.5 para todo: aún no
aprende.")

m = len(X_train)

# 1) Forward
Z = X_train @ W + b
A = sigmoid(Z)

# 2) Loss
error = calcular_error(y_col, A)

# 3) Backpropagation (regla de la cadena)
dZ = A - y_col                      # dL/dZ  para BCE + Sigmoide
dW = X_train.T @ dZ / m             # dL/dW
db = np.mean(dZ)                    # dL/db

# 4) Descenso de gradiente
learning_rate = 0.1
W_nuevo = W - learning_rate * dW
b_nuevo = b - learning_rate * db

print("Pérdida antes de actualizar      :", round(float(error), 6))
print("dZ (primeros 5)                  :", np.round(dZ.ravel()[:5], 5))
print("dW = X^T dZ / m                  :", np.round(dW.ravel(), 6))
print("db                               :", round(float(db), 6))
print()
print("W antes:", np.round(W.ravel(), 6), "-> W después:", np.round(W_nuevo.ravel(), 6))
print("b antes:", round(float(b), 6), "        -> b después:", round(float(b_nuevo), 6))

# Verificación: la pérdida debe disminuir tras el paso
A2 = sigmoid(X_train @ W_nuevo + b_nuevo)
print()
print("Pérdida después de actualizar    :", round(float(calcular_error(y_col, A2)), 6))
print("¿Disminuyó?", calcular_error(y_col, A2) < error)

np.random.seed(42)
W = np.random.randn(3, 1) * 0.01
b = 0.0
learning_rate = 0.1
epochs = 2000

historial_1capa = []
y_col = y_train.reshape(-1, 1)

for epoch in range(epochs):
    # 1. Forward propagation
    Z = X_train @ W + b
    A = sigmoid(Z)

    # 2. Loss
    error = float(calcular_error(y_col, A))
    historial_1capa.append(error)

    # 3. Backpropagation
    m = len(X_train)
    dZ = A - y_col
    dW = X_train.T @ dZ / m
    db = np.mean(dZ)

    # 4. Actualización de parámetros
    W = W - learning_rate * dW
    b = b - learning_rate * db
```

```
    if epoch % 400 == 0:
        print(f"Época {epoch:>5} | Loss {error:.6f}")

print(f"Época {epochs-1:>5} | Loss {historial_1capa[-1]:.6f}")
print()
print("Pesos finales W =", np.round(W.ravel(), 4))
print("Bias final    b =", round(float(b), 4))

W_1capa, b_1capa = W.copy(), float(b)

fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))

axes[0].plot(historial_1capa, color="#2980b9", lw=2)
axes[0].set_title("Figura 4. Curva de pérdida - red de una capa")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("Binary Cross-Entropy")
axes[0].set_yscale("log")

axes[1].plot(historial_1capa[:200], color="#c0392b", lw=2)
axes[1].set_title("Detalle de las primeras 200 épocas (escala lineal)")
axes[1].set_xlabel("Época"); axes[1].set_ylabel("Loss")

plt.tight_layout(); plt.savefig("fig03_loss_una_capa.png", bbox_inches="tight"); plt.show()

print("Pérdida inicial:", round(historial_1capa[0], 6))
print("Pérdida final  :", round(historial_1capa[-1], 6))
print("Reducción      :", f"{(1 - historial_1capa[-1]/historial_1capa[0])*100:.2f} %")

probabilidades = sigmoid(X_test @ W_1capa + b_1capa)
predicciones = (probabilidades >= 0.5).astype(int).ravel()
acc_1capa = accuracy_score(y_test, predicciones)

print("=== RED DE UNA CAPA (Sigmoide + BCE + backpropagation) ===")
print(f"{'Fruta':>6} {'P(banano)':>10} {'Predicción':>12} {'Real':>12} {'Resultado':>10}")
for i in range(len(X_test)):
    pred = "BANANO" if predicciones[i] == 1 else "MANZANA"
    real = "BANANO" if y_test[i] == 1 else "MANZANA"
    ok = "ACIERTO" if pred == real else "ERROR"
    print(f"{i+1:>6} {probabilidades[i,0]:>10.4f} {pred:>12} {real:>12} {ok:>10}")

print()
print("Accuracy en prueba:", round(acc_1capa, 4))
print("Matriz de confusión:\n", confusion_matrix(y_test, predicciones))

def inicializar_parametros(entrada, ocultas, salida, semilla=42):
    """Inicialización de He para capas con ReLU."""
    np.random.seed(semilla)
    W1 = np.random.randn(entrada, ocultas) * np.sqrt(2.0 / entrada)
    b1 = np.zeros((1, ocultas))
    W2 = np.random.randn(ocultas, salida) * np.sqrt(2.0 / ocultas)
    b2 = np.zeros((1, salida))
    return W1, b1, W2, b2


W1, b1, W2, b2 = inicializar_parametros(3, 4, 1)

print("W1:", W1.shape, "-> conecta 3 entradas con 4 neuronas ocultas")
print("b1:", b1.shape)
print("W2:", W2.shape, "-> conecta 4 neuronas ocultas con 1 salida")
print("b2:", b2.shape)
print()
print("W1 =\n", np.round(W1, 4))
print("W2 =", np.round(W2.ravel(), 4))
print()
print("Total de parámetros entrenables:", W1.size + b1.size + W2.size + b2.size)
```

```
y_col = y_train.reshape(-1, 1)

Z1 = X_train @ W1 + b1
A1 = relu(Z1)
Z2 = A1 @ W2 + b2
A2 = sigmoid(Z2)

print("Activaciones de la capa oculta A1 (forma", A1.shape, "):")
print(np.round(A1[:6], 4))
print()
print("Neuronas 'muertas' (siempre en 0) en la capa oculta:",
      int((A1.max(axis=0) == 0).sum()), "de 4")
print()
print("Primeras 5 probabilidades de salida:", np.round(A2.ravel()[:5], 4))
print("Etiquetas reales                   :", y_train[:5])
print()
print("Pérdida inicial (BCE):", round(float(calcular_error(y_col, A2)), 4))

m = len(X_train)

# ---- Forward ----
Z1 = X_train @ W1 + b1
A1 = relu(Z1)
Z2 = A1 @ W2 + b2
A2 = sigmoid(Z2)
error = calcular_error(y_col, A2)

# ---- Capa de salida ----
dZ2 = A2 - y_col
dW2 = A1.T @ dZ2 / m
db2 = np.mean(dZ2, axis=0, keepdims=True)

# ---- Propagación hacia la capa oculta ----
dA1 = dZ2 @ W2.T                      # el error se reparte según W2
dZ1 = dA1 * derivada_relu(Z1)         # se aplica la derivada de ReLU
dW1 = X_train.T @ dZ1 / m
db1 = np.mean(dZ1, axis=0, keepdims=True)

print("Pérdida actual                 :", round(float(error), 6))
print()
print("dZ2 (primeros 5)               :", np.round(dZ2.ravel()[:5], 5))
print("dW2 (gradientes capa de salida) :", np.round(dW2.ravel(), 6))
print("db2                            :", np.round(db2.ravel(), 6))
print()
print("dA1 (error que llega a la capa oculta), primeras 3 filas:\n", np.round(dA1[:3], 6))
print("dZ1 tras multiplicar por ReLU'(Z1), primeras 3 filas:\n", np.round(dZ1[:3], 6))
print("dW1 (gradientes capa oculta):\n", np.round(dW1, 6))
print("db1                            :", np.round(db1.ravel(), 6))
print()
print("Norma del gradiente |dW1| =", round(float(np.linalg.norm(dW1)), 6))
print("Norma del gradiente |dW2| =", round(float(np.linalg.norm(dW2)), 6))

np.random.seed(42)
W1, b1, W2, b2 = inicializar_parametros(3, 4, 1, semilla=42)
learning_rate = 0.05
epochs = 3000
historial_relu = []

for epoch in range(epochs):
    # 1. Forward propagation
    Z1 = X_train @ W1 + b1
    A1 = relu(Z1)
    Z2 = A1 @ W2 + b2
    A2 = sigmoid(Z2)
```

```
    # 2. Loss
    error = float(calcular_error(y_col, A2))
    historial_relu.append(error)

    # 3. Backpropagation
    m = len(X_train)
    dZ2 = A2 - y_col
    dW2 = A1.T @ dZ2 / m
    db2 = np.mean(dZ2, axis=0, keepdims=True)

    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * derivada_relu(Z1)
    dW1 = X_train.T @ dZ1 / m
    db1 = np.mean(dZ1, axis=0, keepdims=True)

    # 4. Actualización (descenso de gradiente)
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    if epoch % 500 == 0:
        print(f"Época {epoch:>5} | Loss {error:.6f}")

print(f"Época {epochs-1:>5} | Loss {historial_relu[-1]:.6f}")

W1_relu, b1_relu, W2_relu, b2_relu = W1.copy(), b1.copy(), W2.copy(), b2.copy()
print()
print("W1 final =\n", np.round(W1_relu, 4))
print("W2 final =", np.round(W2_relu.ravel(), 4))

def forward_multicapa(X_datos, W1, b1, W2, b2, act_oculta=relu):
    """Propagación hacia adelante de la red 3 -> 4 -> 1."""
    Z1 = X_datos @ W1 + b1
    A1 = act_oculta(Z1)
    Z2 = A1 @ W2 + b2
    return sigmoid(Z2), A1, Z1


probs_multi, A1_test, Z1_test = forward_multicapa(X_test, W1_relu, b1_relu, W2_relu, b2_relu)
pred_multi = (probs_multi >= 0.5).astype(int).ravel()
acc_multi = accuracy_score(y_test, pred_multi)

print("=== RED MULTICAPA 3 -> 4 -> 1 (ReLU + Sigmoide) ===")
print(f"{'Fruta':>6} {'P(banano)':>10} {'Predicción':>12} {'Real':>12} {'Resultado':>10}")
for i in range(len(X_test)):
    pred = "BANANO" if pred_multi[i] == 1 else "MANZANA"
    real = "BANANO" if y_test[i] == 1 else "MANZANA"
    ok = "ACIERTO" if pred == real else "ERROR"
    print(f"{i+1:>6} {probs_multi[i,0]:>10.6f} {pred:>12} {real:>12} {ok:>10}")

print()
print("Accuracy en prueba:", round(acc_multi, 4))
print("Matriz de confusión:\n", confusion_matrix(y_test, pred_multi))
print()
print("Activaciones de la capa oculta en prueba (qué neurona responde a cada fruta):")
print(np.round(A1_test, 4))

def entrenar_multicapa(act_oculta, derivada_act, learning_rate, epochs, semilla=42):
    """Entrena la red 3->4->1 con la activación indicada y devuelve historial y parámetros."""
    W1, b1, W2, b2 = inicializar_parametros(3, 4, 1, semilla=semilla)
    historial = []
    for _ in range(epochs):
        Z1 = X_train @ W1 + b1
        A1 = act_oculta(Z1)
```

```
        Z2 = A1 @ W2 + b2
        A2 = sigmoid(Z2)
        historial.append(float(calcular_error(y_col, A2)))

        m = len(X_train)
        dZ2 = A2 - y_col
        dW2 = A1.T @ dZ2 / m
        db2 = np.mean(dZ2, axis=0, keepdims=True)
        dA1 = dZ2 @ W2.T
        dZ1 = dA1 * derivada_act(Z1)
        dW1 = X_train.T @ dZ1 / m
        db1 = np.mean(dZ1, axis=0, keepdims=True)

        W1 -= learning_rate * dW1
        b1 -= learning_rate * db1
        W2 -= learning_rate * dW2
        b2 -= learning_rate * db2
    return historial, (W1, b1, W2, b2)


def derivada_sigmoid(z):
    s = sigmoid(z)
    return s * (1 - s)


hist_relu, params_relu = entrenar_multicapa(relu, derivada_relu, 0.05, 3000)
hist_sig, params_sig = entrenar_multicapa(sigmoid, derivada_sigmoid, 0.05, 3000)

print(f"{'Época':>7} {'Loss ReLU':>12} {'Loss Sigmoide':>15}")
for e in [0, 500, 1000, 1500, 2000, 2500, 2999]:
    print(f"{e:>7} {hist_relu[e]:>12.6f} {hist_sig[e]:>15.6f}")

print()
print("Pérdida final  ReLU    :", round(hist_relu[-1], 6))
print("Pérdida final  Sigmoide:", round(hist_sig[-1], 6))

# Norma del gradiente en la capa oculta al inicio del entrenamiento (misma semilla)
W1i, b1i, W2i, b2i = inicializar_parametros(3, 4, 1, semilla=42)
Z1 = X_train @ W1i + b1i
A1r, A1s = relu(Z1), sigmoid(Z1)
Z2r, Z2s = A1r @ W2i + b2i, A1s @ W2i + b2i
dZ2 = sigmoid(Z2r) - y_col
dA1 = dZ2 @ W2i.T

norma_relu = float(np.linalg.norm(X_train.T @ (dA1 * derivada_relu(Z1)) / len(X_train)))
norma_sig = float(np.linalg.norm(X_train.T @ (dA1 * derivada_sigmoid(Z1)) / len(X_train)))

print("Norma ||dW1|| con ReLU en la capa oculta    :", f"{norma_relu:.6e}")
print("Norma ||dW1|| con Sigmoide en la capa oculta:", f"{norma_sig:.6e}")
print("Razón ReLU / Sigmoide                       :", f"{norma_relu / norma_sig:.2f}x")

fig, axes = plt.subplots(1, 2, figsize=(12.5, 4.2))

axes[0].plot(hist_relu, label="Capa oculta ReLU", color="#27ae60", lw=2)
axes[0].plot(hist_sig, label="Capa oculta Sigmoide", color="#8e44ad", lw=2)
axes[0].plot(historial_1capa, label="Red de una capa (Sigmoide)", color="#2980b9", lw=1.6, ls="--")
axes[0].set_yscale("log")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("Loss (escala log)")
axes[0].set_title("Figura 5. Comparación de curvas de aprendizaje")
axes[0].legend(fontsize=8)

# Fronteras de decisión de los tres modelos
media, desv = scaler.mean_, scaler.scale_
i_f, j_f = 1, 2
g1 = np.linspace(X[:, i_f].min() - 1, X[:, i_f].max() + 1, 300)
```

```
g2 = np.linspace(X[:, j_f].min() - 0.1, X[:, j_f].max() + 0.1, 300)
G1, G2 = np.meshgrid(g1, g2)
pts = np.column_stack([np.full(G1.size, media[0]), G1.ravel(), G2.ravel()])
pts_e = (pts - media) / desv

modelos = [("Una capa (Sigmoide)", lambda Xe: sigmoid(Xe @ W_1capa + b_1capa).ravel()),
           ("Multicapa ReLU", lambda Xe: forward_multicapa(Xe, *params_relu)[0].ravel()),
           ("Multicapa Sigmoide", lambda Xe: forward_multicapa(Xe, *params_sig, act_oculta=sigmoid)
[0].ravel())]

ax = axes[1]
for nombre, fn in modelos:
    Zg = fn(pts_e).reshape(G1.shape)
    ax.contour(G1, G2, Zg, levels=[0.5], linewidths=2)
for clase, nombre, c in [(0, "Manzana", "#c0392b"), (1, "Banano", "#f1c40f")]:
    msk = y == clase
    ax.scatter(X[msk, i_f], X[msk, j_f], c=c, edgecolors="k", s=60, label=nombre)
ax.set_xlabel("Longitud (cm)"); ax.set_ylabel("Nivel de amarillo")
ax.set_title("Figura 6. Fronteras de decisión (curvas = umbral 0.5)")
ax.legend(fontsize=8)

plt.tight_layout(); plt.savefig("fig04_comparacion_activaciones.png", bbox_inches="tight");
plt.show()

def evaluar_modelo(nombre, fn_prob):
    probs = fn_prob(X_test).ravel()
    preds = (probs >= 0.5).astype(int)
    acc = accuracy_score(y_test, preds)
    loss_test = float(calcular_error(y_test.reshape(-1, 1), probs.reshape(-1, 1)))
    return nombre, loss_test, acc


resultados = []
resultados.append(evaluar_modelo("Una capa · Sigmoide", lambda Xe: sigmoid(Xe @ W_1capa +
b_1capa)))
resultados.append(evaluar_modelo("Multicapa · ReLU oculta", lambda Xe: forward_multicapa(Xe,
*params_relu)[0]))
resultados.append(evaluar_modelo("Multicapa · Sigmoide oculta", lambda Xe: forward_multicapa(Xe,
*params_sig, act_oculta=sigmoid)[0]))

print(f"{'Modelo':<32}{'Loss prueba':>13}{'Accuracy':>11}")
print("-" * 56)
for nombre, loss_t, acc in resultados:
    print(f"{nombre:<32}{loss_t:>13.6f}{acc:>11.4f}")
```

Nota. El código se presenta a un espacio por tratarse de material informático. La ejecución completa con sus salidas y gráficas se conserva en los notebooks del repositorio (Salazar Cardenas, 2026) y en las evidencias que lo acompañan.

Apéndice C

Código Fuente del Notebook 03: Clasificación Binaria con TensorFlow y Keras

```
import os
# Ambiente silencioso y determinista para TensorFlow (no altera los resultados)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["PYTHONHASHSEED"] = "0"

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import tensorflow as tf
from tensorflow import keras
import random

random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow:", tf.__version__)
print("Keras     :", keras.__version__)
print("Backend   :", keras.backend.backend())
print("GPU disponible:", bool(tf.config.list_physical_devices('GPU')))

X = np.array([
    # MANZANAS  (clase 0): [peso g, longitud cm, nivel de amarillo 0-1]
    [150, 7.5, 0.10], [160, 8.0, 0.15], [145, 7.2, 0.05],
    [170, 8.3, 0.20], [155, 7.8, 0.10], [165, 8.1, 0.15],
    [140, 7.0, 0.05], [175, 8.5, 0.20], [158, 7.9, 0.10],
    [162, 8.2, 0.15],

    # BANANOS   (clase 1)
    [120, 18.0, 0.90], [125, 19.0, 0.95], [115, 17.5, 0.85],
    [130, 20.0, 1.00], [118, 18.5, 0.90], [123, 19.5, 0.95],
    [110, 17.0, 0.80], [128, 20.5, 1.00], [121, 18.8, 0.90],
    [126, 19.2, 0.95]
], dtype=float)

y = np.array([0] * 10 + [1] * 10)

print("Matriz de características X:", X.shape, "-> (20 frutas, 3 características)")
print("Vector de etiquetas   y:", y.shape, "-> (0 = manzana, 1 = banano)")
print()
print("Primeras 3 manzanas:\n", X[:3])
print("Primeros 3 bananos:\n", X[10:13])
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 80 % entrenamiento / 20 % prueba, conservando la proporción de clases
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# El escalado se AJUSTA solo con entrenamiento y se APLICA a prueba
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("X_train:", X_train.shape, "| X_test:", X_test.shape)
print("y_train:", y_train.shape, "| y_test:", y_test.shape)
print("Distribución de clases en prueba:", np.bincount(y_test))
print()
print("Media de cada característica en entrenamiento (debe ser ~0):")
print(np.round(X_train.mean(axis=0), 6))
print("Desviación estándar (debe ser ~1):")
print(np.round(X_train.std(axis=0), 6))


# Keras espera las etiquetas de clasificación binaria como matriz columna (m, 1)
y_train_k = y_train.reshape(-1, 1).astype("float32")
y_test_k = y_test.reshape(-1, 1).astype("float32")

print("X_train:", X_train.shape, X_train.dtype)
print("y_train:", y_train_k.shape, y_train_k.dtype)
print("X_test :", X_test.shape, "| y_test:", y_test_k.shape)

tf.random.set_seed(42)

modelo = keras.Sequential([
    keras.layers.Input(shape=(3,)),
    keras.layers.Dense(4, activation="relu", name="oculta_relu"),
    keras.layers.Dense(1, activation="sigmoid", name="salida_sigmoid")
])

modelo.summary()

total_params = modelo.count_params()
print("Parámetros entrenables declarados por Keras:", total_params)
print("Parámetros de la implementación NumPy       :", 3*4 + 4 + 4*1 + 1)
print("Coinciden:", total_params == (3*4 + 4 + 4*1 + 1))

modelo.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.01),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

print("Optimizador:", modelo.optimizer.__class__.__name__)
print("Pérdida    :", modelo.loss)
print("Métricas   :", modelo.metrics_names)

historial = modelo.fit(
    X_train,
    y_train_k,
    epochs=200,
    batch_size=4,
    verbose=0
)
```

```
print("Error inicial (época 1)  :", round(historial.history["loss"][0], 6))
print("Error final   (época 200):", round(historial.history["loss"][-1], 6))
print()
print(f"{'Época':>7}{'Loss':>12}{'Accuracy':>11}")
for e in [0, 24, 49, 99, 149, 199]:
    print(f"{e+1:>7}{historial.history['loss'][e]:>12.6f}{historial.history['accuracy']
[e]:>11.4f}")

fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))

axes[0].plot(historial.history["loss"], color="#c0392b", lw=2)
axes[0].set_title("Figura 7. Pérdida durante el entrenamiento (Keras)")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("binary_crossentropy")
axes[0].set_yscale("log")

axes[1].plot(historial.history["accuracy"], color="#2980b9", lw=2)
axes[1].set_title("Figura 8. Accuracy durante el entrenamiento")
axes[1].set_xlabel("Época"); axes[1].set_ylabel("Accuracy")
axes[1].set_ylim(0.4, 1.05)

plt.tight_layout(); plt.savefig("fig05_keras_frutas_binario.png", bbox_inches="tight"); plt.show()

test_loss, test_accuracy = modelo.evaluate(X_test, y_test_k, verbose=0)

print("=== RESULTADOS EN EL CONJUNTO DE PRUEBA ===")
print("Error (binary_crossentropy):", round(float(test_loss), 6))
print("Accuracy                   :", round(float(test_accuracy), 4))
print()

probabilidades = modelo.predict(X_test, verbose=0)
predicciones = (probabilidades >= 0.5).astype(int).ravel()

print(f"{'Fruta':>6}{'P(banano)':>12}{'Predicción':>13}{'Real':>12}{'Resultado':>11}")
for i in range(len(X_test)):
    pred = "BANANO" if predicciones[i] == 1 else "MANZANA"
    real = "BANANO" if y_test[i] == 1 else "MANZANA"
    ok = "ACIERTO" if pred == real else "ERROR"
    print(f"{i+1:>6}{probabilidades[i,0]:>12.6f}{pred:>13}{real:>12}{ok:>11}")

fruta_nueva = np.array([[124.0, 19.0, 0.94]])
fruta_nueva_escalada = scaler.transform(fruta_nueva)

probabilidad = modelo.predict(fruta_nueva_escalada, verbose=0)
resultado = "BANANO" if probabilidad[0][0] >= 0.5 else "MANZANA"

print("Fruta nueva (peso, longitud, amarillo):", fruta_nueva[0])
print("Valores escalados                     :", np.round(fruta_nueva_escalada[0], 4))
print("Probabilidad de ser banano            :", round(float(probabilidad[0][0]), 6))
print("La red piensa que es                  :", resultado)

W1_k, b1_k = modelo.get_layer("oculta_relu").get_weights()
W2_k, b2_k = modelo.get_layer("salida_sigmoid").get_weights()

print("Pesos de la capa oculta W1 (3x4):")
print(np.round(W1_k, 4))
print("Bias de la capa oculta b1:", np.round(b1_k.ravel(), 4))
print()
print("Pesos de la salida W2 (4x1):", np.round(W2_k.ravel(), 4))
print("Bias de la salida  b2       :", round(float(b2_k.ravel()[0]), 4))
print()
print("Activaciones ReLU de la capa oculta para las frutas de prueba:")


def relu(z):
    return np.maximum(0.0, z)
```

```
print(np.round(relu(X_test @ W1_k + b1_k), 4))
print("-> Las neuronas en 0 están inactivas para esa fruta; las positivas aportan evidencia.")
```

Nota. El código se presenta a un espacio por tratarse de material informático. La ejecución completa con sus salidas y gráficas se conserva en los notebooks del repositorio (Salazar Cardenas, 2026) y en las evidencias que lo acompañan.

Apéndice D

Código Fuente del Notebook 04: Clasificación Multiclase con TensorFlow y Keras

```
import os
# Ambiente silencioso y determinista para TensorFlow (no altera los resultados)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["PYTHONHASHSEED"] = "0"

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, ConfusionMatrixDisplay)
import random

random.seed(42); np.random.seed(42); tf.random.set_seed(42)
print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)

X_multi = np.array([
    # MANZANAS - clase 0
    [150, 7.5, .10], [160, 8.0, .15], [145, 7.2, .05], [170, 8.3, .20], [155, 7.8, .10],
    [165, 8.1, .15], [140, 7.0, .05], [175, 8.5, .20], [158, 7.9, .10], [162, 8.2, .15],

    # BANANOS - clase 1
    [120, 18.0, .90], [125, 19.0, .95], [115, 17.5, .85], [130, 20.0, 1.00], [118, 18.5, .90],
    [123, 19.5, .95], [110, 17.0, .80], [128, 20.5, 1.00], [121, 18.8, .90], [126, 19.2, .95],

    # NARANJAS - clase 2
    [180, 8.0, .35], [190, 8.5, .40], [175, 7.8, .30], [200, 9.0, .45], [185, 8.2, .35],
    [195, 8.7, .40], [170, 7.5, .30], [205, 9.2, .45], [188, 8.4, .35], [192, 8.6, .40]
], dtype=float)

y_multi = np.array([0] * 10 + [1] * 10 + [2] * 10)
nombres_frutas = ["MANZANA", "BANANO", "NARANJA"]

print("X_multi:", X_multi.shape, "-> (30 frutas, 3 características)")
print("y_multi:", y_multi.shape, "-> clases:", np.unique(y_multi))
print("Ejemplos por clase:", np.bincount(y_multi))
print()
for c, n in enumerate(nombres_frutas):
    m = y_multi == c
    print(f"{n:<9} peso {X_multi[m,0].mean():6.1f} g | longitud {X_multi[m,1].mean():5.2f} cm |
```

```
amarillo {X_multi[m,2].mean():.2f}")

# División estratificada 80/20 y escalado
X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_test_split(
    X_multi, y_multi, test_size=0.2, random_state=42, stratify=y_multi
)

scaler_multi = StandardScaler()
X_train_multi = scaler_multi.fit_transform(X_train_multi)
X_test_multi = scaler_multi.transform(X_test_multi)

print("X_train_multi:", X_train_multi.shape, "| X_test_multi:", X_test_multi.shape)
print("Clases en entrenamiento:", np.bincount(y_train_multi))
print("Clases en prueba       :", np.bincount(y_test_multi))
print("Etiquetas de prueba    :", y_test_multi)

# Verificación del comportamiento de Softmax
z_ejemplo = np.array([[2.0, 1.0, 0.1]])
exp = np.exp(z_ejemplo)
softmax_manual = exp / exp.sum()

print("Logits z            :", z_ejemplo[0])
print("Softmax manual      :", np.round(softmax_manual[0], 4), "| suma:",
round(softmax_manual.sum(), 6))
print("Clase elegida argmax:", int(np.argmax(softmax_manual)))
print()
z_otro = np.array([[0.4, 2.8, 1.1]])
s_otro = np.exp(z_otro) / np.exp(z_otro).sum()
print("Logits z            :", z_otro[0])
print("Softmax             :", np.round(s_otro[0], 4), "| argmax:", int(np.argmax(s_otro)))
print("-> Softmax conserva el orden de los logits y los normaliza para que sumen 1.")

tf.random.set_seed(42)

modelo_multi = keras.Sequential([
    keras.layers.Input(shape=(3,)),
    keras.layers.Dense(6, activation="relu", name="oculta"),
    keras.layers.Dense(3, activation="softmax", name="salida_softmax")
])

modelo_multi.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.01),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

modelo_multi.summary()

historial_multi = modelo_multi.fit(
    X_train_multi,
    y_train_multi,
    epochs=300,
    batch_size=4,
    validation_split=0.2,
    verbose=0
)

print("Error inicial:", round(historial_multi.history["loss"][0], 6))
print("Error final  :", round(historial_multi.history["loss"][-1], 6))
print()
print(f"{'Época':>7}{'Loss':>11}{'Acc':>8}{'Val loss':>11}{'Val acc':>9}")
for e in [0, 49, 99, 149, 199, 249, 299]:
    h = historial_multi.history
    print(f"{e+1:>7}{h['loss'][e]:>11.5f}{h['accuracy'][e]:>8.3f}"
          f"{h['val_loss'][e]:>11.5f}{h['val_accuracy'][e]:>9.3f}")
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))
h = historial_multi.history

axes[0].plot(h["loss"], label="Entrenamiento", color="#c0392b", lw=2)
axes[0].plot(h["val_loss"], label="Validación", color="#2980b9", lw=1.6, ls="--")
axes[0].set_yscale("log")
axes[0].set_title("Figura 9. Pérdida multiclase")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("sparse_categorical_crossentropy")
axes[0].legend(fontsize=8)

axes[1].plot(h["accuracy"], label="Entrenamiento", color="#27ae60", lw=2)
axes[1].plot(h["val_accuracy"], label="Validación", color="#8e44ad", lw=1.6, ls="--")
axes[1].set_ylim(0.2, 1.05)
axes[1].set_title("Figura 10. Accuracy multiclase")
axes[1].set_xlabel("Época"); axes[1].set_ylabel("Accuracy")
axes[1].legend(fontsize=8)

plt.tight_layout(); plt.savefig("fig06_keras_multiclase_curvas.png", bbox_inches="tight");
plt.show()

loss_multi, accuracy_multi = modelo_multi.evaluate(X_test_multi, y_test_multi, verbose=0)

probabilidades_multi = modelo_multi.predict(X_test_multi, verbose=0)
predicciones_multi = np.argmax(probabilidades_multi, axis=1)
acc_manual = accuracy_score(y_test_multi, predicciones_multi)

print("=== CLASIFICACIÓN MULTICLASE (manzana / banano / naranja) ===")
print("Loss en prueba  :", round(float(loss_multi), 6))
print("Accuracy (Keras):", round(float(accuracy_multi), 4))
print("Accuracy (sklearn, verificación):", round(acc_manual, 4))
print()
print(f"{'#':>3} {'P(manzana)':>11} {'P(banano)':>10} {'P(naranja)':>11}   {'Predicción':>10}
{'Real':>10} {'Resultado':>10}")
for i in range(len(X_test_multi)):
    p = probabilidades_multi[i]
    pred, real = nombres_frutas[predicciones_multi[i]], nombres_frutas[y_test_multi[i]]
    ok = "ACIERTO" if pred == real else "ERROR"
    print(f"{i+1:>3} {p[0]:>11.4f} {p[1]:>10.4f} {p[2]:>11.4f}   {pred:>10} {real:>10} {ok:>10}")
print()
print("Cada fila de probabilidades suma 1:",
      bool(np.allclose(probabilidades_multi.sum(axis=1), 1.0)))

cm = confusion_matrix(y_test_multi, predicciones_multi)
print("Matriz de confusión (filas = reales, columnas = predichas):")
print(cm)
print()
print(classification_report(y_test_multi, predicciones_multi,
                            target_names=nombres_frutas, zero_division=0))

fig, ax = plt.subplots(figsize=(5.2, 4.4))
ConfusionMatrixDisplay(cm, display_labels=nombres_frutas).plot(
    ax=ax, cmap="Blues", colorbar=False, values_format="d")
ax.set_title("Figura 11. Matriz de confusión - frutas multiclase")
plt.tight_layout(); plt.savefig("fig07_confusion_multiclase.png", bbox_inches="tight"); plt.show()

media, desv = scaler_multi.mean_, scaler_multi.scale_
i_f, j_f = 1, 2   # longitud vs nivel de amarillo

g1 = np.linspace(X_multi[:, i_f].min() - 1.0, X_multi[:, i_f].max() + 1.0, 320)
g2 = np.linspace(X_multi[:, j_f].min() - 0.10, X_multi[:, j_f].max() + 0.10, 320)
G1, G2 = np.meshgrid(g1, g2)
pts = np.column_stack([np.full(G1.size, media[0]), G1.ravel(), G2.ravel()])
pts_e = (pts - media) / desv
```

```
probs_grid = modelo_multi.predict(pts_e, verbose=0)
clase_grid = np.argmax(probs_grid, axis=1).reshape(G1.shape)
conf_grid = probs_grid.max(axis=1).reshape(G1.shape)

fig, axes = plt.subplots(1, 2, figsize=(13, 4.6))
colores_clase = ["#c0392b", "#f1c40f", "#e67e22"]
import matplotlib.colors as mcolors
cmap = mcolors.ListedColormap(["#f5b7b1", "#fcf3cf", "#fad7a0"])

axes[0].contourf(G1, G2, clase_grid, levels=[-0.5, 0.5, 1.5, 2.5], cmap=cmap, alpha=0.9)
for c, n in enumerate(nombres_frutas):
    m = y_multi == c
    axes[0].scatter(X_multi[m, i_f], X_multi[m, j_f], c=colores_clase[c],
                    edgecolors="k", s=70, label=n)
axes[0].set_xlabel("Longitud (cm)"); axes[0].set_ylabel("Nivel de amarillo")
axes[0].set_title("Figura 12. Regiones de decisión (Softmax + argmax)")
axes[0].legend(fontsize=8)

im = axes[1].pcolormesh(G1, G2, conf_grid, cmap="viridis", vmin=0.33, vmax=1.0, shading="auto")
for c, n in enumerate(nombres_frutas):
    m = y_multi == c
    axes[1].scatter(X_multi[m, i_f], X_multi[m, j_f], c=colores_clase[c],
                    edgecolors="w", s=70, label=n)
axes[1].set_xlabel("Longitud (cm)"); axes[1].set_ylabel("Nivel de amarillo")
axes[1].set_title("Figura 13. Confianza de la predicción (probabilidad máxima)")
axes[1].legend(fontsize=8, loc="lower right")
fig.colorbar(im, ax=axes[1], label="P(clase predicha)")

plt.tight_layout(); plt.savefig("fig08_frontera_multiclase.png", bbox_inches="tight"); plt.show()

print("Probabilidad máxima promedio en la malla:", round(float(conf_grid.mean()), 4))
print("Zonas con confianza < 0.60 (fronteras difusas):",
      f"{(conf_grid < 0.60).mean()*100:.2f} % del área")

fruta_nueva_multi = np.array([[190.0, 8.5, 0.38]])
fruta_escalada_multi = scaler_multi.transform(fruta_nueva_multi)

prob = modelo_multi.predict(fruta_escalada_multi, verbose=0)
clase = int(np.argmax(prob, axis=1)[0])

print("Fruta nueva (peso, longitud, amarillo):", fruta_nueva_multi[0])
print("Probabilidades por clase:")
for n, p in zip(nombres_frutas, prob[0]):
    barra = "" * int(round(p * 30))
    print(f"   {n:<9} {p:.4f}  {barra}")
print()
print("Clase predicha (argmax):", clase)
print("La red piensa que es     :", nombres_frutas[clase])
```

Nota. El código se presenta a un espacio por tratarse de material informático. La ejecución completa con sus salidas y gráficas se conserva en los notebooks del repositorio (Salazar Cardenas, 2026) y en las evidencias que lo acompañan.

Apéndice E

Código Fuente del Notebook 05: MNIST con TensorFlow y Keras

```
import os
# Ambiente silencioso y determinista para TensorFlow (no altera los resultados)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["PYTHONHASHSEED"] = "0"

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import tensorflow as tf
from tensorflow import keras
import random, time, gc
from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay

random.seed(42); np.random.seed(42); tf.random.set_seed(42)
print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)
print("Backend    :", keras.backend.backend())
print("Dispositivos:", [d.device_type for d in tf.config.list_physical_devices()])

ruta_mnist = keras.utils.get_file(
    "mnist.npz",
    "https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz",
)

with np.load(ruta_mnist, allow_pickle=False) as datos:
    x_train_mnist = datos["x_train"]                    # uint8 (60000, 28, 28)
    y_train_mnist = datos["y_train"]                    # uint8 (60000,)
    x_test_mnist = datos["x_test"].astype("float32") / 255.0   # normalizado [0,1]
    y_test_mnist = datos["y_test"]

print("Archivo:", ruta_mnist)
print()
print("Entrenamiento:", x_train_mnist.shape, y_train_mnist.shape, x_train_mnist.dtype)
print("Prueba       :", x_test_mnist.shape, y_test_mnist.shape, x_test_mnist.dtype)
print("Rango de píxeles (entrenamiento):", x_train_mnist.min(), "-", x_train_mnist.max())
print("Rango de píxeles (prueba)       :", x_test_mnist.min(), "-", x_test_mnist.max())
print("Clases:", np.unique(y_train_mnist))
print("Ejemplos por clase:", np.bincount(y_train_mnist))
print()
print("Primer ejemplo: etiqueta real =", y_train_mnist[0], "| forma =", x_train_mnist[0].shape)
print("Memoria del conjunto de entrenamiento:",
      round(x_train_mnist.nbytes / 1024**2, 1), "MB (uint8)")

fig, axes = plt.subplots(2, 10, figsize=(11.5, 2.8))
for fila in range(2):
    for col in range(10):
        idx = fila * 10 + col
```

```
        axes[fila, col].imshow(x_train_mnist[idx], cmap="gray")
        axes[fila, col].set_title(str(y_train_mnist[idx]), fontsize=9)
        axes[fila, col].axis("off")
fig.suptitle("Figura 14. Primeras 20 imágenes de entrenamiento de MNIST con su etiqueta real",
y=1.06)
plt.tight_layout(); plt.savefig("fig09_mnist_muestra.png"); plt.show()
plt.close(fig)
_ = gc.collect()

def normalizar_lote(imagenes):
    """Convierte un lote uint8 (n,28,28) a float32 (n,784) en [0,1]."""
    return imagenes.reshape(len(imagenes), -1).astype("float32") / 255.0


lote_demo = normalizar_lote(x_train_mnist[:5])
print("Lote de 5 imágenes ->", lote_demo.shape, lote_demo.dtype)
print("Mínimo:", lote_demo.min(), "| Máximo:", lote_demo.max())
print()
print("Primeros 12 valores del vector de 784 píxeles (zona de borde, casi todo ceros):")
print(np.round(lote_demo[0][:12], 3))
print()
zona = lote_demo[0].reshape(28, 28)[10:18, 8:20]
print("Bloque central 8x12 de la misma imagen (donde sí hay trazo):")
print(np.round(zona, 2))
print()
print("Verificación: 784 =", 28 * 28, "->", lote_demo.shape[1] == 28 * 28)

tf.random.set_seed(42)

modelo_mnist = keras.Sequential([
    keras.layers.Input(shape=(784,)),
    keras.layers.Dense(128, activation="relu", name="oculta_128"),
    keras.layers.Dense(10, activation="softmax", name="salida_10")
])

modelo_mnist.summary()

tipos = [capa.__class__.__name__ for capa in modelo_mnist.layers]
print()
print("Capas del modelo            :", tipos)
print("¿Hay alguna capa convolucional?", any("Conv" in t for t in tipos))
print("¿Hay alguna capa de pooling?  ", any("Pooling" in t for t in tipos))
print("Parámetros entrenables      :", f"{modelo_mnist.count_params():,}")
print("-> Verificación de la restricción: red 100 % densa (perceptrón multicapa).")

perdida_fn = keras.losses.SparseCategoricalCrossentropy()
optimizador = keras.optimizers.Adam(learning_rate=0.001)
metrica_acc = keras.metrics.SparseCategoricalAccuracy()

print("Pérdida    :", perdida_fn.__class__.__name__)
print("Optimizador:", optimizador.__class__.__name__, "| lr =", optimizador.learning_rate)


@tf.function
def paso_entrenamiento(xb, yb):
    """Un paso completo: forward -> loss -> backpropagation -> actualización."""
    with tf.GradientTape() as cinta:
        logits = modelo_mnist(xb, training=True)
        perdida = perdida_fn(yb, logits)
    gradientes = cinta.gradient(perdida, modelo_mnist.trainable_variables)   # backpropagation
    optimizador.apply_gradients(zip(gradientes, modelo_mnist.trainable_variables))  # actualización
    return perdida


def evaluar_conjunto(x_datos, y_datos, tam_lote=1000, normalizar=False):
```

```
    """Evalúa pérdida y accuracy por lotes, sin materializar todas las predicciones."""
    perdidas, aciertos, total = 0.0, 0, 0
    clases, confianzas = [], []
    for inicio in range(0, len(x_datos), tam_lote):
        xb = x_datos[inicio:inicio + tam_lote]
        yb = y_datos[inicio:inicio + tam_lote]
        if normalizar:
            xb = normalizar_lote(xb)
        else:
            xb = xb.reshape(len(xb), -1)
        probs = modelo_mnist(xb, training=False).numpy()
        perdidas += float(perdida_fn(yb, probs)) * len(yb)
        aciertos += int((probs.argmax(axis=1) == yb).sum())
        total += len(yb)
        clases.append(probs.argmax(axis=1))
        confianzas.append(probs.max(axis=1))
    return (perdidas / total, aciertos / total,
            np.concatenate(clases), np.concatenate(confianzas))


print()
print("Funciones de entrenamiento y evaluación definidas correctamente")

xb0 = normalizar_lote(x_train_mnist[:1])
yb0 = y_train_mnist[:1]
probs0 = modelo_mnist(xb0, training=False).numpy()

print("Etiqueta real de la imagen 0:", yb0[0])
print("Probabilidades sin entrenar :", np.round(probs0[0], 4))
print("Suma de probabilidades    :", round(float(probs0[0].sum()), 6))
print("Pérdida inicial             :", round(float(perdida_fn(yb0, probs0)), 4))
print("Referencia teórica -ln(1/10):", round(-np.log(0.1), 4))

BATCH_SIZE = 128
EPOCHS = 5
N_VALIDACION = 6000
n_entrenamiento = len(x_train_mnist) - N_VALIDACION

indices_train = np.arange(n_entrenamiento)
indices_val = np.arange(n_entrenamiento, len(x_train_mnist))

historial = {"loss": [], "accuracy": [], "val_loss": [], "val_accuracy": []}
t0 = time.time()

for epoch in range(EPOCHS):
    # ---------- ENTRENAMIENTO ----------
    np.random.shuffle(indices_train)
    perdidas_epoca, aciertos_epoca, vistos = 0.0, 0, 0

    for inicio in range(0, n_entrenamiento, BATCH_SIZE):
        idx = indices_train[inicio:inicio + BATCH_SIZE]
        xb = normalizar_lote(x_train_mnist[idx])
        yb = y_train_mnist[idx]

        perdida = paso_entrenamiento(xb, yb)          # forward + loss + backward + update

        probs = modelo_mnist(xb, training=False).numpy()
        perdidas_epoca += float(perdida) * len(idx)
        aciertos_epoca += int((probs.argmax(axis=1) == yb).sum())
        vistos += len(idx)

    # ---------- VALIDACIÓN ----------
    perd_val, aciertos_val, total_val = 0.0, 0, 0
    for inicio in range(0, N_VALIDACION, 512):
        idx = indices_val[inicio:inicio + 512]
```

```
        xb = normalizar_lote(x_train_mnist[idx])
        yb = y_train_mnist[idx]
        probs = modelo_mnist(xb, training=False).numpy()
        perd_val += float(perdida_fn(yb, probs)) * len(yb)
        aciertos_val += int((probs.argmax(axis=1) == yb).sum())
        total_val += len(yb)

    historial["loss"].append(perdidas_epoca / vistos)
    historial["accuracy"].append(aciertos_epoca / vistos)
    historial["val_loss"].append(perd_val / total_val)
    historial["val_accuracy"].append(aciertos_val / total_val)
    gc.collect()

tiempo = time.time() - t0
print(f"{'Época':>6}{'Loss':>10}{'Accuracy':>10}{'Val loss':>11}{'Val acc':>10}")
for i in range(EPOCHS):
    print(f"{i+1:>6}{historial['loss'][i]:>10.4f}{historial['accuracy'][i]:>10.4f}"
          f"{historial['val_loss'][i]:>11.4f}{historial['val_accuracy'][i]:>10.4f}")
print()
print(f"Tiempo total: {tiempo:.1f} s ({tiempo/EPOCHS:.1f} s por época)")
print(f"Lotes por época: {int(np.ceil(n_entrenamiento / BATCH_SIZE))}")

fig, axes = plt.subplots(1, 2, figsize=(10.5, 3.6))
epocas = range(1, EPOCHS + 1)

axes[0].plot(epocas, historial["loss"], marker="o", label="Entrenamiento", color="#c0392b", lw=2)
axes[0].plot(epocas, historial["val_loss"], marker="s", label="Validación", color="#2980b9", lw=2,
ls="--")
axes[0].set_title("Figura 15. Error durante el entrenamiento")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("Loss"); axes[0].legend(fontsize=8)

axes[1].plot(epocas, historial["accuracy"], marker="o", label="Entrenamiento", color="#27ae60",
lw=2)
axes[1].plot(epocas, historial["val_accuracy"], marker="s", label="Validación", color="#8e44ad",
lw=2, ls="--")
axes[1].set_ylim(0.88, 1.0)
axes[1].set_title("Figura 16. Precisión durante el entrenamiento")
axes[1].set_xlabel("Época"); axes[1].set_ylabel("Accuracy"); axes[1].legend(fontsize=8)

plt.tight_layout(); plt.savefig("fig10_mnist_curvas_keras.png"); plt.show()
plt.close(fig)
_ = gc.collect()

test_loss, test_accuracy, clases_test, confianzas_test = evaluar_conjunto(
    x_test_mnist, y_test_mnist, tam_lote=1000
)

print("=== RESULTADOS MNIST · TensorFlow + Keras ===")
print("Error de prueba    :", round(test_loss, 6))
print("Accuracy de prueba :", round(test_accuracy, 6))
print("Imágenes correctas :", int(round(test_accuracy * len(y_test_mnist))), "de",
len(y_test_mnist))
print("Imágenes erróneas  :", len(y_test_mnist) - int(round(test_accuracy * len(y_test_mnist))))
print()
print("Confianza promedio en aciertos:",
      round(float(confianzas_test[clases_test == y_test_mnist].mean()), 4))
print("Confianza promedio en errores :",
      round(float(confianzas_test[clases_test != y_test_mnist].mean()), 4))
print("-> El modelo está mucho menos seguro cuando se equivoca: la probabilidad máxima es")
print("   una señal útil para decidir cuándo requerir revisión humana.")

probs10 = modelo_mnist(x_test_mnist[:10].reshape(10, 784),
                                    training=False).numpy()

print("=== PRIMERAS 10 IMÁGENES DE PRUEBA ===")
```

```
print(f"{'Imagen':>7}{'Real':>6}{'Predicción':>12}{'Confianza':>11}{'Resultado':>11}")
for i in range(10):
    prediccion = int(np.argmax(probs10[i]))
    conf = float(probs10[i].max())
    ok = "ACIERTO" if prediccion == y_test_mnist[i] else "ERROR"
    print(f"{i+1:>7}{int(y_test_mnist[i]):>6}{prediccion:>12}{conf:>11.4f}{ok:>11}")

print()
print("Vector completo de probabilidades de la imagen 1:")
print(np.round(probs10[0], 6), "-> suma:", round(float(probs10[0].sum()), 6))

fig, axes = plt.subplots(2, 5, figsize=(11.5, 3.0))
for i, ax in enumerate(axes.ravel()):
    pred = int(np.argmax(probs10[i]))
    real = int(y_test_mnist[i])
    ax.imshow(x_test_mnist[i].reshape(28, 28), cmap="gray")
    ax.set_title(f"Real: {real} · Pred: {pred}", fontsize=9,
                 color="#27ae60" if pred == real else "#c0392b")
    ax.axis("off")
fig.suptitle("Figura 17. Predicciones sobre las primeras 10 imágenes de prueba", y=1.06)
plt.tight_layout(); plt.savefig("fig11_mnist_predicciones.png"); plt.show()
plt.close(fig)
_ = gc.collect()

errores = np.where(clases_test != y_test_mnist)[0]

print("Cantidad total de errores:", len(errores), "de", len(y_test_mnist))
print("Tasa de error            :", f"{len(errores)/len(y_test_mnist)*100:.2f} %")
print()
print("Primeros 15 errores (índice | real -> predicho | confianza):")
for idx in errores[:15]:
    print(f"  {idx:>5} | {y_test_mnist[idx]} -> {clases_test[idx]} | {confianzas_test[idx]:.4f}")

cm = confusion_matrix(y_test_mnist, clases_test)
cm_norm = cm.astype("float") / cm.sum(axis=1, keepdims=True)

fig, ax = plt.subplots(figsize=(5.6, 4.6))
ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(
    ax=ax, cmap="Blues", colorbar=True, values_format="d", xticks_rotation=0)
ax.set_title("Figura 18. Matriz de confusión - MNIST (10 000 imágenes)")
plt.tight_layout(); plt.savefig("fig12_mnist_confusion.png"); plt.show()
plt.close(fig)
_ = gc.collect()

fig, ax = plt.subplots(figsize=(5.6, 4.6))
ConfusionMatrixDisplay(cm_norm, display_labels=range(10)).plot(
    ax=ax, cmap="Blues", colorbar=True, values_format=".3f", xticks_rotation=0)
ax.set_title("Figura 19. Matriz normalizada por clase real (recall)")
plt.tight_layout(); plt.savefig("fig12b_mnist_confusion_norm.png"); plt.show()
plt.close(fig)
_ = gc.collect()

print(classification_report(y_test_mnist, clases_test,
                           target_names=[str(d) for d in range(10)], digits=4))

pares = sorted([(cm[r, c], r, c) for r in range(10) for c in range(10) if r != c], reverse=True)
print("Top 5 confusiones (real -> predicho):")
for cant, r, c in pares[:5]:
    print(f"   {r} -> {c} : {cant} casos ({cant/cm[r].sum()*100:.2f} % de los dígitos '{r}')")
print()
print("Clase con mejor recall :", int(np.argmax(np.diag(cm_norm))),
      f"({np.diag(cm_norm).max():.4f})")
print("Clase con peor  recall :", int(np.argmin(np.diag(cm_norm))),
      f"({np.diag(cm_norm).min():.4f})")
```

```
n_mostrar = 9
fig, axes = plt.subplots(1, n_mostrar, figsize=(11.5, 2.2))
for ax, idx in zip(axes, errores[:n_mostrar]):
    ax.imshow(x_test_mnist[idx].reshape(28, 28), cmap="gray")
    ax.set_title(f"Real {y_test_mnist[idx]}\nPred {clases_test[idx]}", fontsize=8, color="#c0392b")
    ax.axis("off")
fig.suptitle("Figura 20. Ejemplos de dígitos clasificados incorrectamente", y=1.14)
plt.tight_layout(); plt.savefig("fig13_mnist_errores.png"); plt.show()
plt.close(fig)
_ = gc.collect()

def entrenar_variante(n_ocultas, x_idx, epochs=2, lr=0.001, semilla=42):
    """Entrena una red 784 -> n_ocultas -> 10 y devuelve métricas y tiempo."""
    tf.random.set_seed(semilla)
    m = keras.Sequential([
        keras.layers.Input(shape=(784,)),
        keras.layers.Dense(n_ocultas, activation="relu"),
        keras.layers.Dense(10, activation="softmax")
    ])
    opt = keras.optimizers.Adam(learning_rate=lr)
    loss_fn = keras.losses.SparseCategoricalCrossentropy()

    @tf.function
    def paso(xb, yb):
        with tf.GradientTape() as cinta:
            perdida = loss_fn(yb, m(xb, training=True))
        g = cinta.gradient(perdida, m.trainable_variables)
        opt.apply_gradients(zip(g, m.trainable_variables))
        return perdida

    idx = np.array(x_idx)
    np.random.shuffle(idx)
    t0 = time.time()
    for _ in range(epochs):
        for i in range(0, len(idx), BATCH_SIZE):
            sel = idx[i:i + BATCH_SIZE]
            paso(normalizar_lote(x_train_mnist[sel]), y_train_mnist[sel])
    dt = time.time() - t0

    loss_t, acc_t, _, _ = evaluar_variante(m, loss_fn)
    params = m.count_params()
    del m
    keras.backend.clear_session()
    gc.collect()
    return params, loss_t, acc_t, dt


def evaluar_variante(m, loss_fn, tam_lote=1000):
    perdidas, aciertos, total = 0.0, 0, 0
    for inicio in range(0, len(x_test_mnist), tam_lote):
        xb = x_test_mnist[inicio:inicio + tam_lote].reshape(tam_lote, -1)
        yb = y_test_mnist[inicio:inicio + tam_lote]
        probs = m(xb, training=False).numpy()
        perdidas += float(loss_fn(yb, probs)) * len(yb)
        aciertos += int((probs.argmax(axis=1) == yb).sum())
        total += len(yb)
    return perdidas / total, aciertos / total, None, None


idx_sub = np.arange(0, n_entrenamiento, 3)     # 18 000 imágenes estratificadas por posición
print("Subconjunto del experimento:", len(idx_sub), "imágenes")
print()
print(f"{'Neuronas':>9}{'Parámetros':>13}{'Test loss':>11}{'Test acc':>10}{'Segundos':>10}")
print("-" * 53)
resultados_cap = []
```

```
for n_ocultas in [32, 128, 256]:
    params, loss_t, acc_t, dt = entrenar_variante(n_ocultas, idx_sub)
    resultados_cap.append((n_ocultas, params, loss_t, acc_t, dt))
    print(f"{n_ocultas:>9}{params:>13,}{loss_t:>11.4f}{acc_t:>10.4f}{dt:>10.1f}")

fig, ax = plt.subplots(figsize=(6.2, 3.8))
ax.plot([r[0] for r in resultados_cap], [r[3] for r in resultados_cap],
        marker="o", lw=2, color="#16a085")
for n, _, _, at, _ in resultados_cap:
    ax.annotate(f"{at:.4f}", (n, at), textcoords="offset points",
                xytext=(0, 9), ha="center", fontsize=9)
ax.set_xticks([32, 128, 256])
ax.set_xlabel("Neuronas en la capa oculta")
ax.set_ylabel("Accuracy en prueba")
ax.set_ylim(min(r[3] for r in resultados_cap) - 0.01, 1.0)
ax.set_title("Figura 21. Accuracy vs capacidad de la capa oculta (2 épocas, 18 000 imágenes)")
plt.tight_layout(); plt.savefig("fig14_mnist_capacidad.png"); plt.show()
plt.close(fig)
_ = gc.collect()
```

Nota. El código se presenta a un espacio por tratarse de material informático. La ejecución completa con sus salidas y gráficas se conserva en los notebooks del repositorio (Salazar Cardenas, 2026) y en las evidencias que lo acompañan.

Apéndice F

Código Fuente del Notebook 06: MNIST con PyTorch

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import time
from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay

torch.manual_seed(42)
print("PyTorch    :", torch.__version__)
print("CUDA disponible:", torch.cuda.is_available())
print("Dispositivo usado:", "cuda" if torch.cuda.is_available() else "cpu")

transform = transforms.ToTensor()      # PIL/numpy -> tensor (1,28,28) en [0,1]

# La primera ejecución descarga MNIST (~11 MB) a la carpeta ./data.
# Se silencia la barra de progreso de la descarga para que la evidencia del
# notebook quede limpia; en Colab la descarga se muestra de forma nativa.
import contextlib
import urllib.request
import torchvision.datasets.utils as tvutils


def _descarga_sin_barra(url, filename, chunk_size=1024 * 256):
    """Descarga equivalente a la de torchvision, pero sin barra de progreso."""
    with urllib.request.urlopen(url) as respuesta, open(filename, "wb") as archivo:
        while True:
            bloque = respuesta.read(chunk_size)
            if not bloque:
                break
            archivo.write(bloque)


@contextlib.contextmanager
def descarga_silenciosa():
    original = tvutils._urlretrieve
    tvutils._urlretrieve = _descarga_sin_barra
    try:
        yield
    finally:
        tvutils._urlretrieve = original


with descarga_silenciosa():
```

```
    train_dataset = datasets.MNIST(root="./data", train=True, download=True,
                                   transform=transform)
    test_dataset = datasets.MNIST(root="./data", train=False, download=True,
                                   transform=transform)


print("Imágenes de entrenamiento:", len(train_dataset))
print("Imágenes de prueba       :", len(test_dataset))

imagen0, etiqueta0 = train_dataset[0]
print()
print("Tensor de la primera imagen:", tuple(imagen0.shape), imagen0.dtype)
print("Rango de píxeles           :", float(imagen0.min()), "-", float(imagen0.max()))
print("Etiqueta real              :", etiqueta0)

BATCH_SIZE = 64

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print("Lotes de entrenamiento:", len(train_loader),
      f"({len(train_dataset)} imágenes / {BATCH_SIZE})")
print("Lotes de prueba       :", len(test_loader))

imagenes_lote, etiquetas_lote = next(iter(train_loader))
print()
print("Forma de un lote de imágenes:", tuple(imagenes_lote.shape), "-> (lote, canales, alto,
ancho)")
print("Forma de un lote de etiquetas:", tuple(etiquetas_lote.shape))
print("Etiquetas del primer lote   :", etiquetas_lote[:16].tolist())

fig, axes = plt.subplots(2, 10, figsize=(11.5, 2.8))
lote_img, lote_etq = next(iter(DataLoader(test_dataset, batch_size=20, shuffle=False)))
for i, ax in enumerate(axes.ravel()):
    ax.imshow(lote_img[i].squeeze(), cmap="gray")
    ax.set_title(str(int(lote_etq[i])), fontsize=9)
    ax.axis("off")
fig.suptitle("Figura 22. Primer lote de prueba cargado con DataLoader (PyTorch)", y=1.04)
plt.tight_layout(); plt.savefig("fig15_pytorch_lote.png"); plt.show()

class RedMNIST(nn.Module):
    """Perceptrón multicapa 784 -> 128 -> 10 (sin capas convolucionales)."""

    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()           # (lote,1,28,28) -> (lote,784)
        self.capa1 = nn.Linear(784, 128)      # capa oculta
        self.relu = nn.ReLU()                 # activación de la capa oculta
        self.salida = nn.Linear(128, 10)      # logits de las 10 clases

    def forward(self, x):
        x = self.flatten(x)
        x = self.capa1(x)
        x = self.relu(x)
        x = self.salida(x)
        return x


modelo_pytorch = RedMNIST()
print(modelo_pytorch)
print()
total = sum(p.numel() for p in modelo_pytorch.parameters())
entrenables = sum(p.numel() for p in modelo_pytorch.parameters() if p.requires_grad)
print("Parámetros totales     :", f"{total:,}")
print("Parámetros entrenables :", f"{entrenables:,}")
```

```
print()
for nombre, p in modelo_pytorch.named_parameters():
    print(f"  {nombre:<18} forma {tuple(p.shape)!s:<14} parámetros {p.numel():>8,}")

modelo_pytorch.eval()
imagen_prueba = test_dataset[0][0].unsqueeze(0)     # (1,28,28)

with torch.no_grad():
    logits = modelo_pytorch(imagen_prueba)
    probabilidades = torch.softmax(logits, dim=1)

print("Entrada (lote, canales, alto, ancho):", tuple(imagen_prueba.shape))
print("Logits de salida                    :", tuple(logits.shape))
print("Logits (sin entrenar)               :", np.round(logits.numpy()[0], 4))
print()
print("Softmax aplicado manualmente para inspección:")
print(np.round(probabilidades.numpy()[0], 4), "| suma:", round(float(probabilidades.sum()), 4))
print("-> Antes de entrenar las probabilidades son casi uniformes (~0.1 por clase).")

criterio = nn.CrossEntropyLoss()

optimizador = optim.Adam(modelo_pytorch.parameters(), lr=0.001)

print("Criterio de pérdida:", criterio)
print("Optimizador        :", optimizador.__class__.__name__, "| lr =",
      optimizador.param_groups[0]["lr"])
print("Número de tensores de parámetros registrados:", len(optimizador.param_groups[0]["params"]))
print()
print("Modelo, criterio y optimizador listos")

epochs = 3
historial_loss = []

t0 = time.time()
for epoch in range(epochs):
    modelo_pytorch.train()              # modo entrenamiento
    error_total = 0.0
    correctas = 0
    total = 0

    for batch_idx, (imagenes, etiquetas) in enumerate(train_loader):
        # 1. Limpiar gradientes anteriores
        optimizador.zero_grad()

        # 2. Forward propagation
        salidas = modelo_pytorch(imagenes)

        # 3. Loss
        error = criterio(salidas, etiquetas)

        # 4. Backpropagation (cálculo de gradientes)
        error.backward()

        # 5. Actualización de pesos
        optimizador.step()

        # --- Métricas de seguimiento ---
        error_total += error.item()
        _, predichas = torch.max(salidas, 1)
        correctas += (predichas == etiquetas).sum().item()
        total += etiquetas.size(0)

        if batch_idx % 200 == 0:
            print(f"  Época {epoch+1}/{epochs} | lote {batch_idx:>4}/{len(train_loader)} "
                  f"| loss del lote: {error.item():.4f}")
```

```
    promedio_error = error_total / len(train_loader)
    acc_epoca = correctas / total
    historial_loss.append((promedio_error, acc_epoca))
    print(f"Época {epoch+1} | Error promedio: {promedio_error:.4f} | Accuracy entrenamiento:
{acc_epoca:.4f}\n")

tiempo = time.time() - t0
print(f"Tiempo total de entrenamiento: {tiempo:.1f} s ({tiempo/epochs:.1f} s por época)")

fig, axes = plt.subplots(1, 2, figsize=(10.5, 3.6))

axes[0].plot([e[0] for e in historial_loss], marker="o", lw=2, color="#c0392b")
axes[0].set_title("Figura 23. Pérdida por época (PyTorch)")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("CrossEntropyLoss")
for i, e in enumerate(historial_loss):
    axes[0].annotate(f"{e[0]:.4f}", (i, e[0]), textcoords="offset points",
                     xytext=(0, 9), ha="center", fontsize=9)

axes[1].plot([e[1] for e in historial_loss], marker="s", lw=2, color="#27ae60")
axes[1].set_ylim(min(e[1] for e in historial_loss) - 0.01, 1.0)
axes[1].set_title("Figura 24. Accuracy de entrenamiento por época")
axes[1].set_xlabel("Época"); axes[1].set_ylabel("Accuracy")
for i, e in enumerate(historial_loss):
    axes[1].annotate(f"{e[1]:.4f}", (i, e[1]), textcoords="offset points",
                     xytext=(0, -16), ha="center", fontsize=9)

plt.tight_layout(); plt.savefig("fig16_pytorch_curvas.png"); plt.show()

# Un único paso para dejar gradientes cargados y poder inspeccionarlos
modelo_pytorch.train()
imagenes, etiquetas = next(iter(train_loader))
optimizador.zero_grad()
error = criterio(modelo_pytorch(imagenes), etiquetas)
error.backward()

print("Pérdida del lote inspeccionado:", round(error.item(), 6))
print()
print(f"{'Parámetro':<22}{'Norma del gradiente':>22}{'Grad. máximo':>15}{'Grad. mínimo':>15}")
for nombre, p in modelo_pytorch.named_parameters():
    g = p.grad
    print(f"{nombre:<22}{float(torch.norm(g)):>22.6f}{float(g.max()):>15.6f}
{float(g.min()):>15.6f}")
print()
print("-> Ningún gradiente es idénticamente cero: la señal de error llegó a todas las capas.")

modelo_pytorch.eval()
correctas = 0
total = 0
todas_pred = []
todas_real = []

with torch.no_grad():
    for imagenes, etiquetas in test_loader:
        salidas = modelo_pytorch(imagenes)
        _, predicciones = torch.max(salidas, 1)
        total += etiquetas.size(0)
        correctas += (predicciones == etiquetas).sum().item()
        todas_pred.extend(predicciones.tolist())
        todas_real.extend(etiquetas.tolist())

accuracy_pytorch = correctas / total
todas_pred = np.array(todas_pred)
todas_real = np.array(todas_real)
```

```
print("=== RESULTADOS MNIST · PyTorch ===")
print("Imágenes evaluadas :", total)
print("Predicciones correctas:", correctas)
print("Predicciones erróneas :", total - correctas)
print("Accuracy en prueba :", round(accuracy_pytorch, 6))
print("Tasa de error      :", f"{(1-accuracy_pytorch)*100:.2f} %")

def predecir_imagen(indice):
    """Devuelve la predicción y las probabilidades de una imagen del conjunto de prueba."""
    imagen, etiqueta_real = test_dataset[indice]
    entrada = imagen.unsqueeze(0)
    modelo_pytorch.eval()
    with torch.no_grad():
        salida = modelo_pytorch(entrada)
        probs = torch.softmax(salida, dim=1)[0]
        prediccion = int(torch.argmax(probs).item())
    return imagen, int(etiqueta_real), prediccion, probs.numpy()


fig, axes = plt.subplots(1, 5, figsize=(11.5, 2.8))
for ax, idx in zip(axes, [0, 1, 2, 3, 4]):
    imagen, real, pred, probs = predecir_imagen(idx)
    ax.imshow(imagen.squeeze(), cmap="gray")
    ax.set_title(f"Real: {real} | Pred: {pred}\nconfianza {probs.max():.4f}",
                 fontsize=9, color="#27ae60" if real == pred else "#c0392b")
    ax.axis("off")
fig.suptitle("Figura 25. Predicciones individuales (PyTorch)", y=1.08)
plt.tight_layout(); plt.savefig("fig17_pytorch_predicciones.png"); plt.show()

imagen, real, pred, probs = predecir_imagen(0)
print("Número real    :", real)
print("Número predicho:", pred)
print()
print("Distribución de probabilidad por dígito:")
for d, p in enumerate(probs):
    print(f"   {d}: {p:.6f}  {'' * int(round(p * 40))}")

cm = confusion_matrix(todas_real, todas_pred)

fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.4))
ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(
    ax=axes[0], cmap="Oranges", colorbar=True, values_format="d", xticks_rotation=0)
axes[0].set_title("Figura 26. Matriz de confusión - MNIST con PyTorch")

cm_norm = cm.astype("float") / cm.sum(axis=1, keepdims=True)
ConfusionMatrixDisplay(cm_norm, display_labels=range(10)).plot(
    ax=axes[1], cmap="Oranges", colorbar=True, values_format=".3f", xticks_rotation=0)
axes[1].set_title("Figura 27. Matriz normalizada por clase real (recall)")
plt.tight_layout(); plt.savefig("fig18_pytorch_confusion.png"); plt.show()

print(classification_report(todas_real, todas_pred,
                            target_names=[str(d) for d in range(10)], digits=4))

pares = sorted([(cm[r, c], r, c) for r in range(10) for c in range(10) if r != c], reverse=True)
print("Top 5 confusiones (real -> predicho):")
for cant, r, c in pares[:5]:
    print(f"   {r} -> {c} : {cant} casos ({cant/cm[r].sum()*100:.2f} % de los '{r}')")

errores_idx = np.where(todas_pred != todas_real)[0]
print("Cantidad de errores:", len(errores_idx))

fig, axes = plt.subplots(1, 9, figsize=(11.5, 2.2))
for ax, idx in zip(axes, errores_idx[:9]):
    imagen, real, pred, _ = predecir_imagen(int(idx))
    ax.imshow(imagen.squeeze(), cmap="gray")
```

```
    ax.set_title(f"Real {real}\nPred {pred}", fontsize=8, color="#c0392b")
    ax.axis("off")
fig.suptitle("Figura 28. Errores del clasificador PyTorch", y=1.12)
plt.tight_layout(); plt.savefig("fig19_pytorch_errores.png"); plt.show()

# Tabla resumen con los resultados REALES obtenidos en esta ejecución
acc_keras_referencia = None   # se completa en la sección de resultados del documento técnico

print("=" * 66)
print("RESUMEN DE RESULTADOS OBTENIDOS (ejecución real de este notebook)")
print("=" * 66)
print(f"Framework            : PyTorch {torch.__version__}")
print(f"Arquitectura         : 784 -> 128 -> 10 (ReLU + Softmax, sin CNN)")
print(f"Parámetros           : {sum(p.numel() for p in modelo_pytorch.parameters()):,}")
print(f"Épocas               : {epochs}")
print(f"Batch size           : {BATCH_SIZE}")
print(f"Loss por época       : {', '.join(f'{e[0]:.4f}' for e in historial_loss)}")
print(f"Acc. entrenamiento   : {historial_loss[-1][1]:.4f}")
print(f"ACCURACY DE PRUEBA   : {accuracy_pytorch:.4f}  ({int(correctas)}/{total})")
print("=" * 66)
```

Nota. El código se presenta a un espacio por tratarse de material informático. La ejecución completa con sus salidas y gráficas se conserva en los notebooks del repositorio (Salazar Cardenas, 2026) y en las evidencias que lo acompañan.